
LAPORAN PRAKTIKUM
MATA KULIAH TEKNOLOGI IOT II

ARSITEKTUR INTERNET OF THINGS DAN MIKROKONTROLER



Oleh:

Nama: Aila Nadianti

NPM : 25782059

Kelas : 3C

Dosen:

Dr. Ir. Septafiansyah Dwi Putra, S.T., M.T.

Agus Ambarwari, S.Pd., M.Kom.

TEKNOLOGI REKAYASA INTERNET
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI LAMPUNG

2026

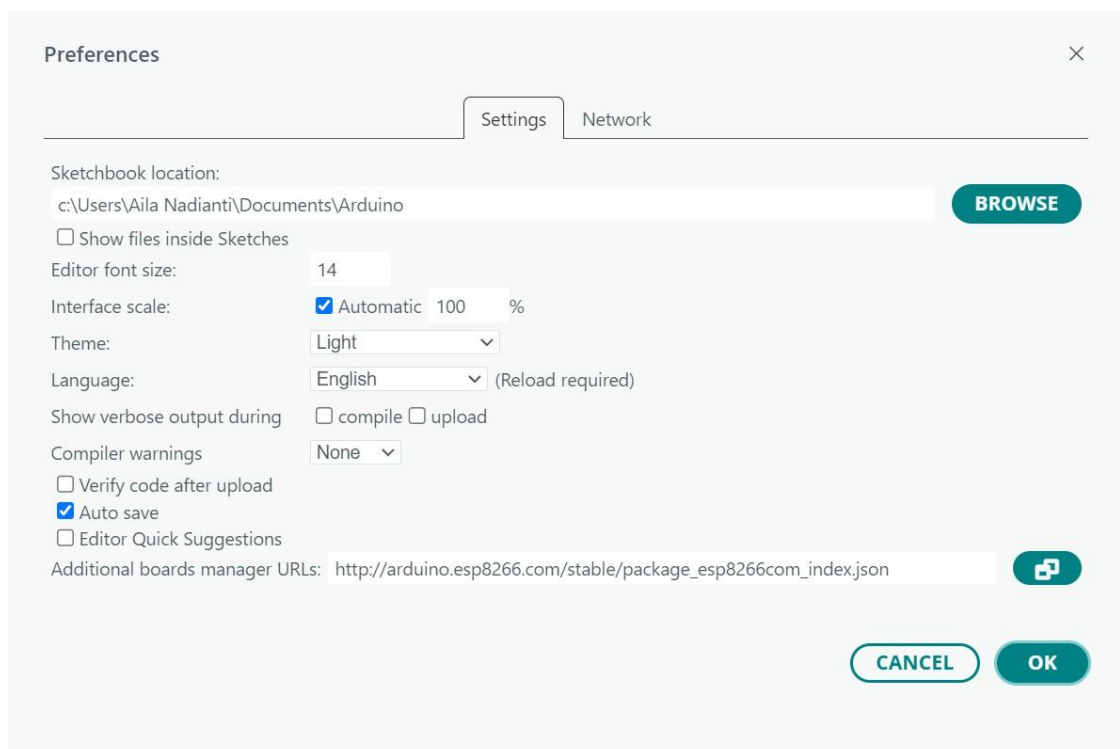
1.4.2 Konfigurasi dan Instalasi Lingkungan Pengembangan (Arduino IDE)

Arduino IDE merupakan perangkat lunak yang digunakan untuk menulis, mengompilasi, dan mengunggah program ke mikrokontroler. Sebelum digunakan untuk pemrograman, Arduino IDE perlu diinstal dan dikonfigurasi terlebih dahulu sesuai dengan board yang akan digunakan. Proses instalasi dimulai dengan mengunduh Arduino IDE melalui situs resmi Arduino sesuai dengan sistem operasi yang digunakan. Setelah file instalasi selesai diunduh, jalankan installer dengan melakukan *double click*. Selanjutnya, ikuti tahapan instalasi yang tersedia, seperti menyetujui ketentuan lisensi, menentukan komponen yang diperlukan, memilih lokasi penyimpanan, kemudian menekan tombol **Install**. Setelah proses instalasi selesai, Arduino IDE dapat dijalankan melalui **Start Menu** maupun *shortcut* pada desktop.

Konfigurasi Board ESP8266

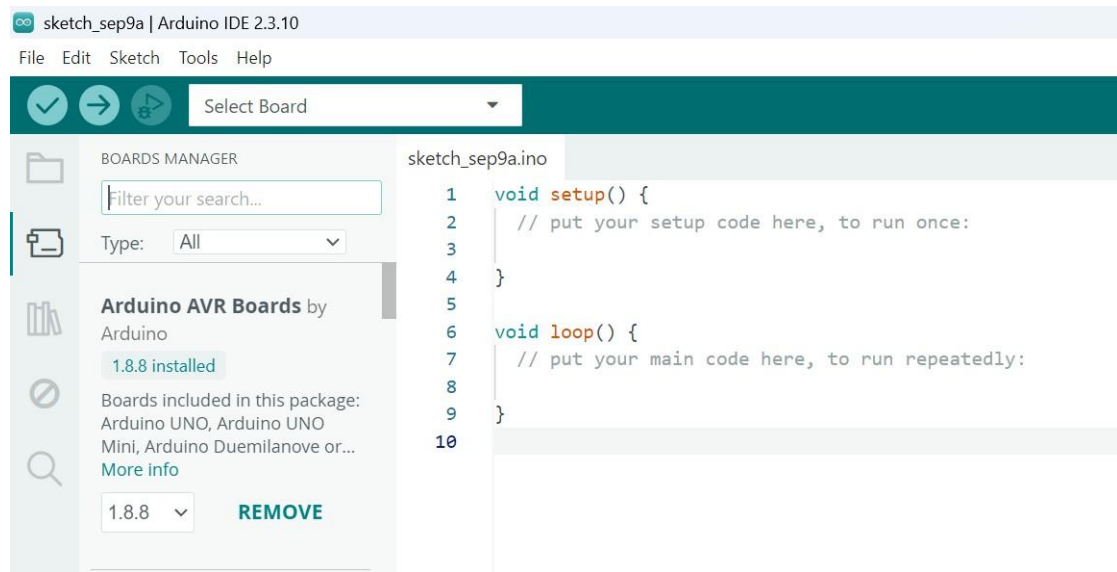
Agar Arduino IDE dapat mengenali dan digunakan untuk memprogram mikrokontroler ESP8266, diperlukan instalasi *board package* ESP8266. Konfigurasi dilakukan dengan langkah-langkah berikut:

1. Jalankan aplikasi **Arduino IDE**.
2. Pilih menu **File > Preferences**.
3. Pada bagian **Additional Boards Manager URLs**, masukkan alamat berikut:
http://arduino.esp8266.com/stable/package_esp8266com_index.json



4. Setelah itu, masuk ke menu **Tools > Board > Boards Manager....**
5. Cari **esp8266** pada kolom pencarian.

6. Pilih paket **esp8266 by ESP8266 Community**, kemudian klik **Install**.
7. Tunggu hingga proses instalasi paket selesai.



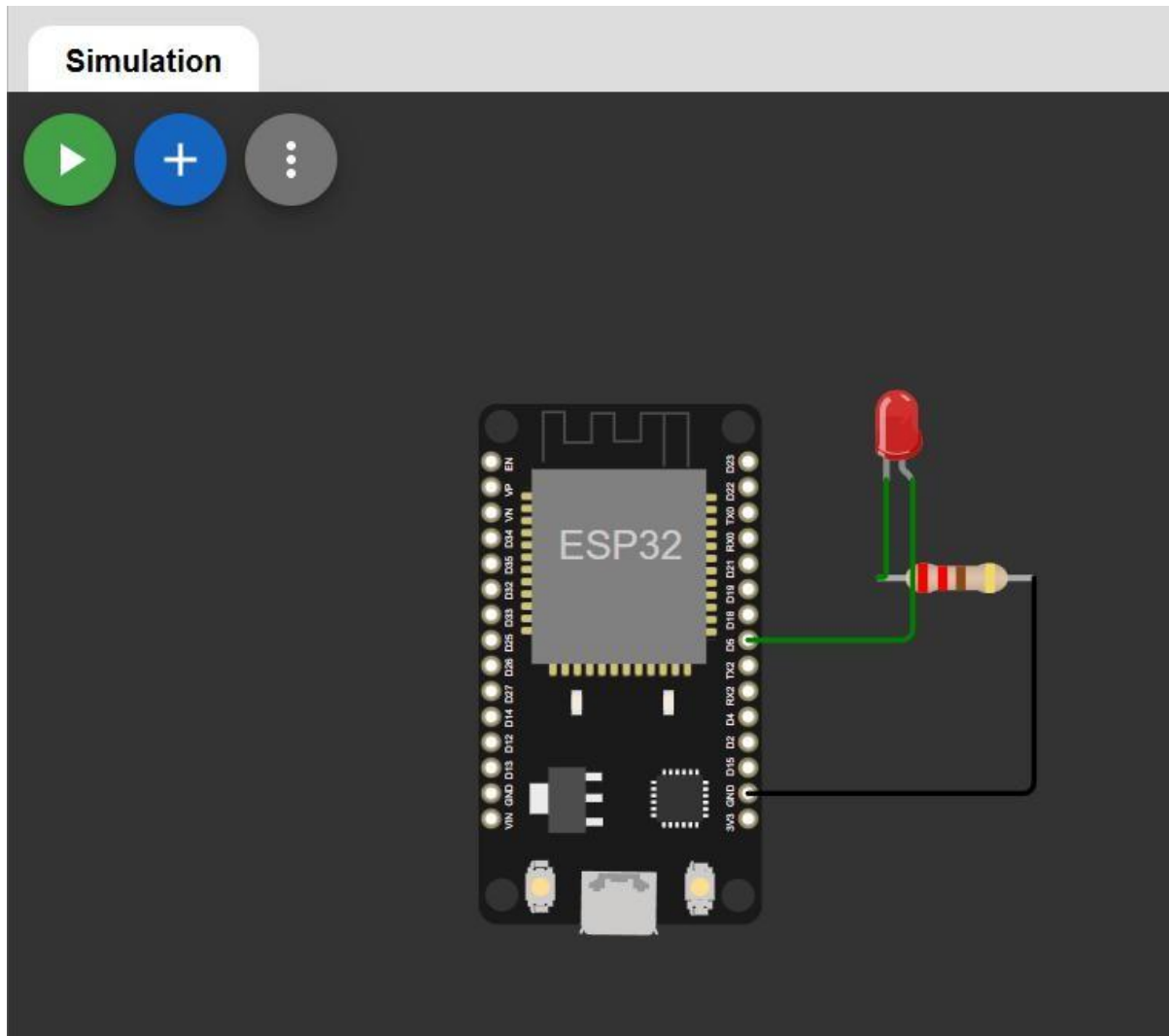
Setelah *board package* berhasil dipasang, lakukan pengaturan board melalui menu **Tools**. Pada bagian **Board**, pilih **NodeMCU 1.0 (ESP-12E Module)**. Kemudian, hubungkan NodeMCU ke laptop menggunakan kabel USB. Setelah perangkat terdeteksi, pilih **Port COM** yang sesuai dengan port yang muncul pada Arduino IDE.

Dengan konfigurasi tersebut, Arduino IDE sudah siap digunakan untuk membuat dan mengunggah program ke board NodeMCU ESP8266.

1.5 Praktikum

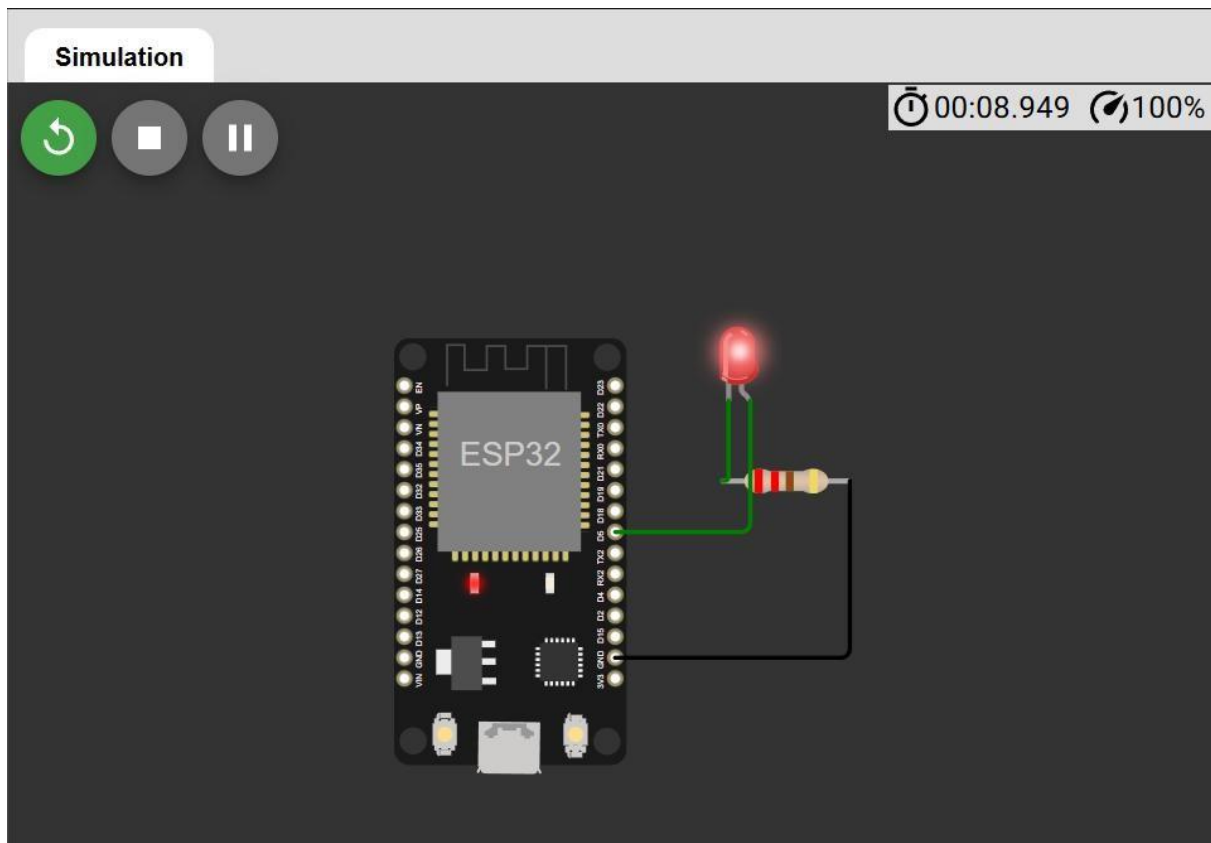
1.5.1 Praktikum 1: Digital Output (Menyalakan LED)

Praktikum ini bertujuan memprogram NodeMCU agar mengalirkan tegangan 3.3V (HIGH) secara terprogram untuk menyalakan komponen eksternal.



Gambar 1. Simulasi rangkaian Digital Output

- a. Pengendali Utama: Board mikrokontroler mengelola eksekusi algoritma dan pendistribusian sinyal keluaran.
- b. Umpan Balik Visual: LED menampilkan status kerja; bernilai HIGH saat menyala dan LOW saat padam.
- c. Proteksi Arus: Resistor berfungsi menahan arus berlebih agar komponen LED tidak terbakar.
- d. Konektivitas Sinyal: Jalur output digital dialokasikan pada pin D1 (GPIO5) untuk mengendalikan LED.
- e. Referensi Ground: Terminal GND menyediakan jalur return arus dan menyamakan acuan ground sistem.



Gambar 2. Kondisi LED saat program dijalankan

Tampilan *screenshot* menunjukkan kondisi LED yang sedang menyala akibat penerimaan nilai logika HIGH dari pin keluaran mikrokontroler. Begitu instruksi program berubah menjadi LOW, arus terhenti dan LED otomatis padam. Mekanisme transmisi sinyal HIGH-LOW ini menjadi dasar utama dalam pengontrolan komponen *actuator* sederhana.

Praktikum 1 - Digital Output Dimulai!

LED Menyala

LED Mati

LED Menyala

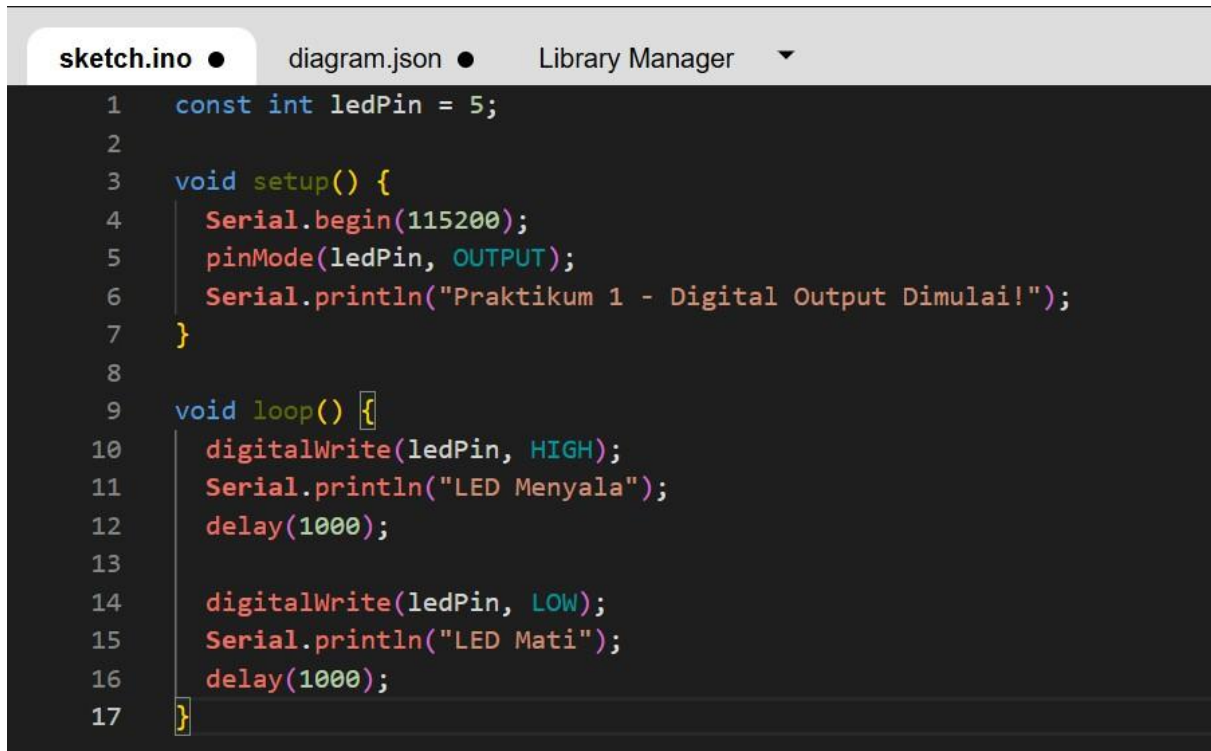
LED Mati

LED Menyala

Gambar 3. Tampilan Serial Monitor Digital Output

Fungsi utama Serial Monitor adalah menampilkan keluaran informasi dari skrip yang sedang dieksekusi mikrokontroler. Dalam pengujian ini, Serial Monitor mencetak pembaruan kondisi

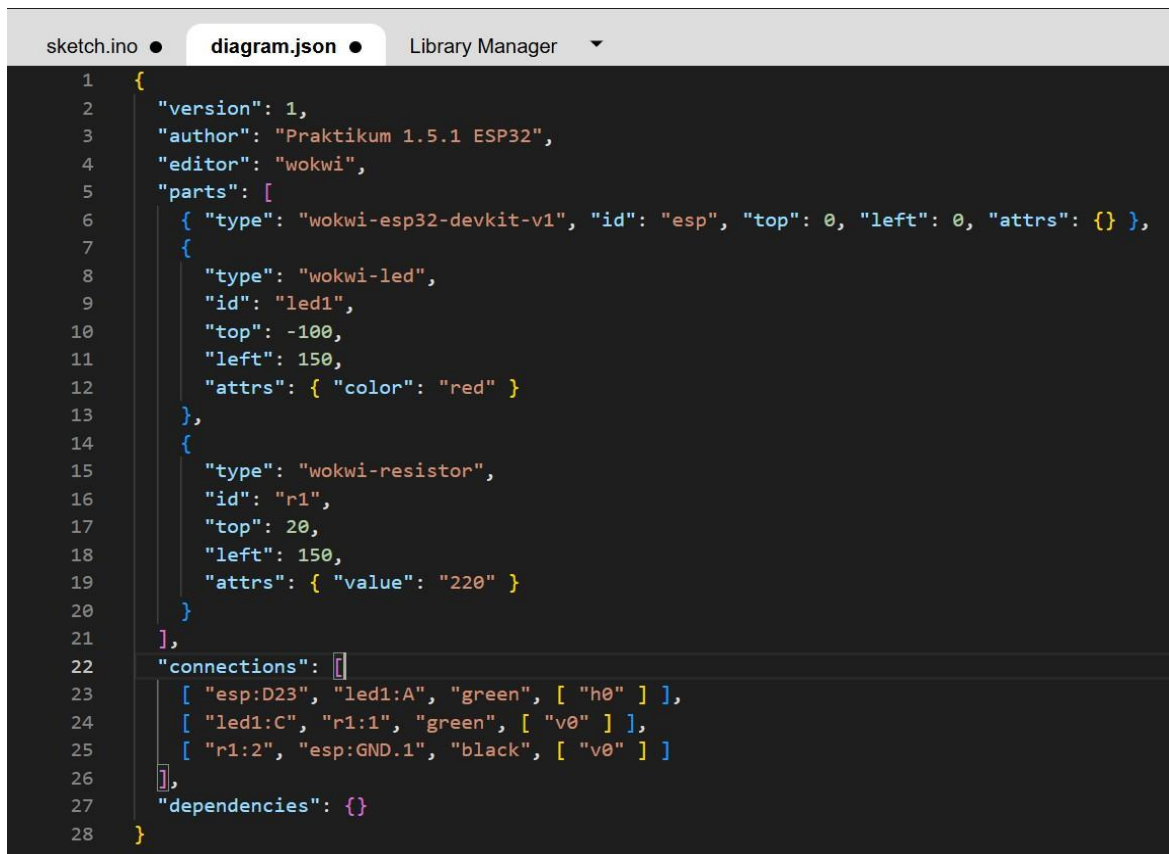
LED secara bergantian (saat menyala dan padam). Tampilan log tersebut menjadi sarana validasi bahwa instruksi program berhasil dijalankan dan transisi logika terjadi sesuai perintah.

The image shows a screenshot of an Arduino IDE window. At the top, there are three tabs: 'sketch.ino' (selected), 'diagram.json', and 'Library Manager'. The main area displays C++ code for controlling an LED. The code is as follows:

```
1  const int ledPin = 5;
2
3  void setup() {
4      Serial.begin(115200);
5      pinMode(ledPin, OUTPUT);
6      Serial.println("Praktikum 1 - Digital Output Dimulai!");
7  }
8
9  void loop() {
10     digitalWrite(ledPin, HIGH);
11     Serial.println("LED Menyala");
12     delay(1000);
13
14     digitalWrite(ledPin, LOW);
15     Serial.println("LED Mati");
16     delay(1000);
17 }
```

Gambar 4. Kode program sketch.ino

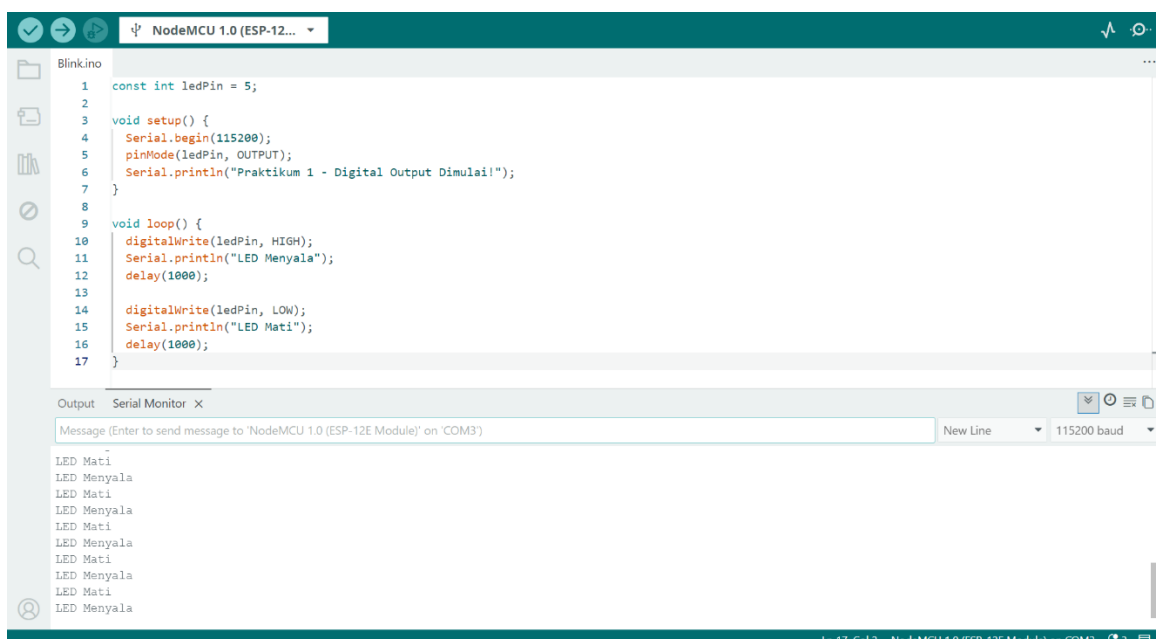
File sketch.ino merupakan bagian yang berisi program untuk mengontrol LED. Program menentukan pin LED sebagai OUTPUT, kemudian memberikan HIGH untuk menyalakan LED dan LOW untuk mematikannya. Perintah delay(1000) membuat setiap kondisi berlangsung sekitar satu detik sehingga LED terlihat berkedip secara teratur.



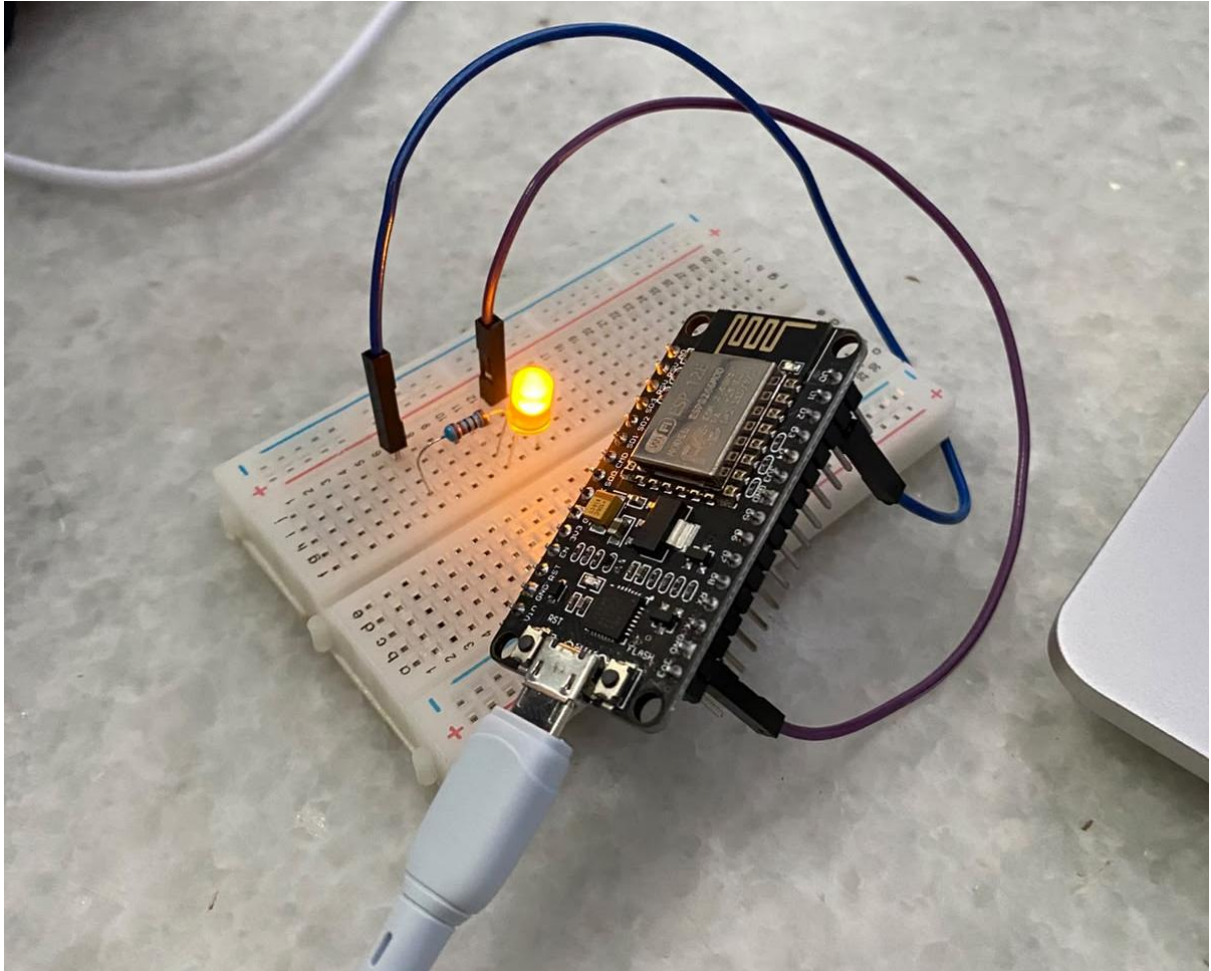
```
1 {
2   "version": 1,
3   "author": "Praktikum 1.5.1 ESP32",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": -100,
11      "left": 150,
12      "attrs": { "color": "red" }
13    },
14    {
15      "type": "wokwi-resistor",
16      "id": "r1",
17      "top": 20,
18      "left": 150,
19      "attrs": { "value": "220" }
20    }
21  ],
22  "connections": [
23    [ "esp:D23", "led1:A", "green", [ "h0" ] ],
24    [ "led1:C", "r1:1", "green", [ "v0" ] ],
25    [ "r1:2", "esp:GND.1", "black", [ "v0" ] ]
26  ],
27  "dependencies": {}
28 }
```

Gambar 5. Konfigurasi diagram.json

Diagram.json itu ibarat denah atau gambar rangkaiannya—di situ didaftarkan semua alat yang dipakai dalam simulasi, mulai dari board mikrokontroler, LED, resistor, sampai kabel-kabel yang menyambungkan antar pin-nya. Jadi sederhananya, kodingan di sketch.ino itu bertindak sebagai "otak" yang ngatur cara kerja rangkaian, sedangkan diagram.json yang nyediain fisik perangkat dan susunannya.



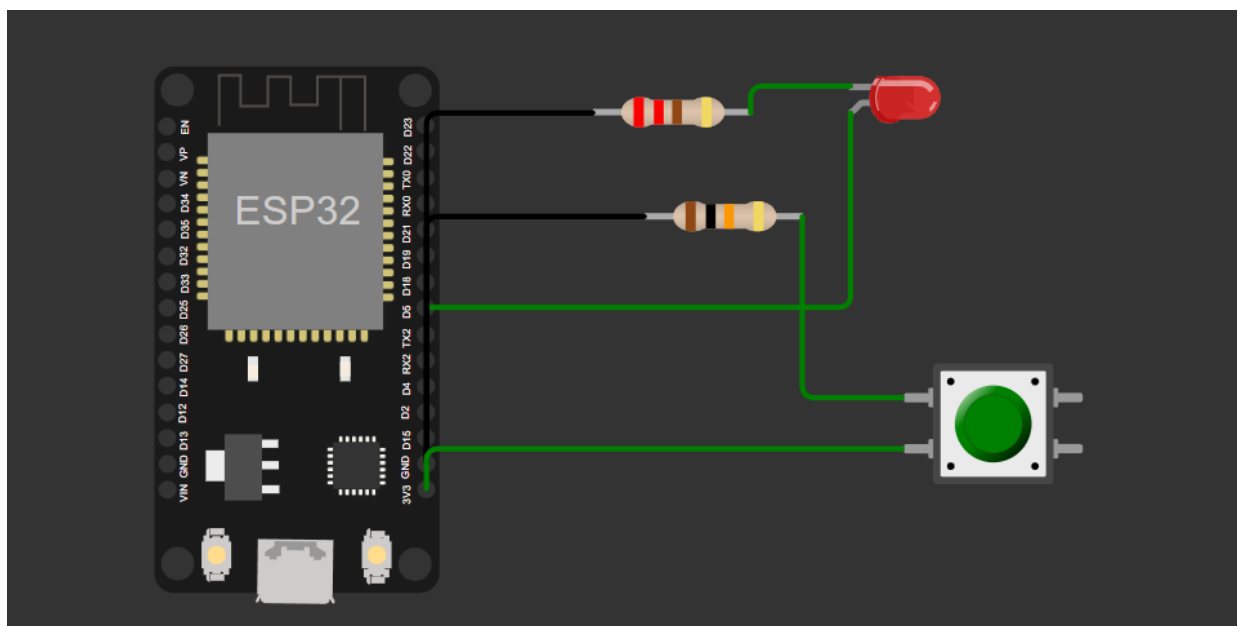
Gambar 6. Simulasi Menggunakan Arduino



Gambar 7. Menggunakan Alat Secara Langsung

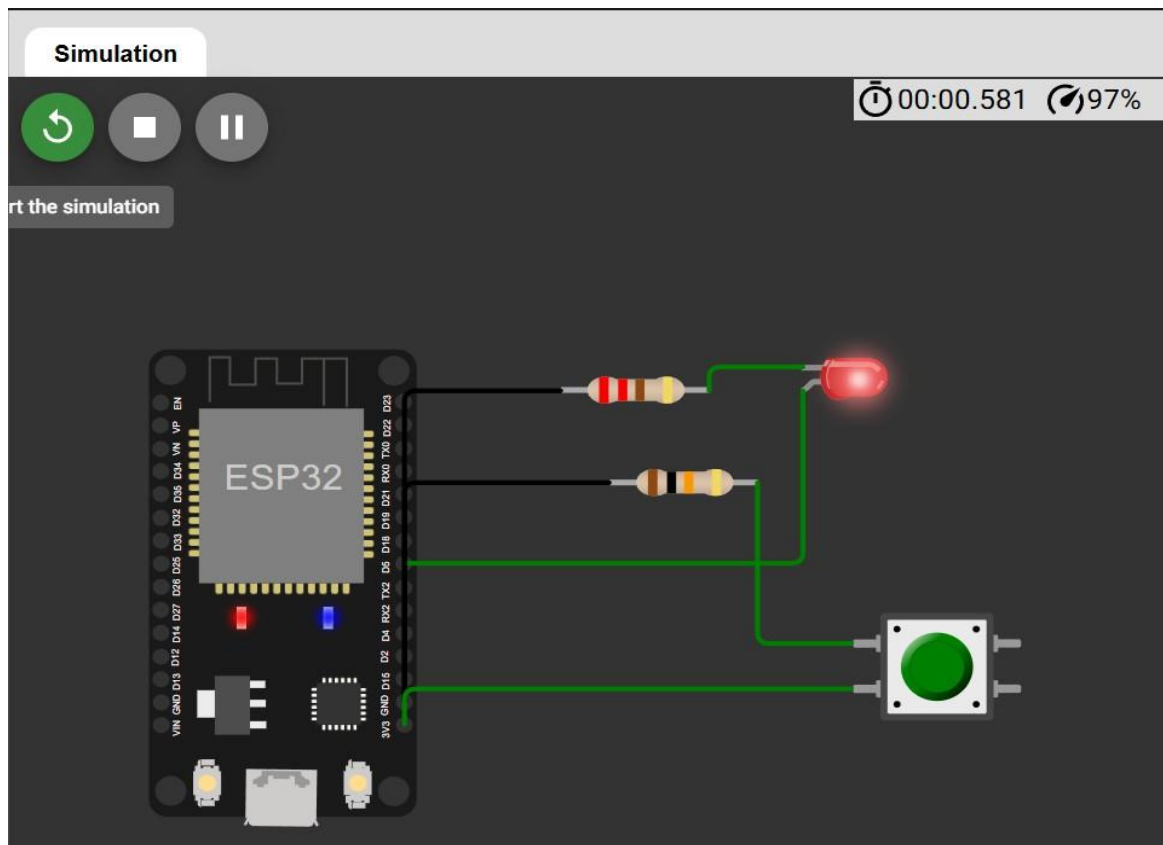
1.5.2 Praktikum 2: Digital Input (Membaca Push Button)

Praktikum ini bertujuan mendeteksi interaksi fisik dengan membaca status tegangan dari sebuah tombol, memanfaatkan rangkaian resistor pull-down eksternal.



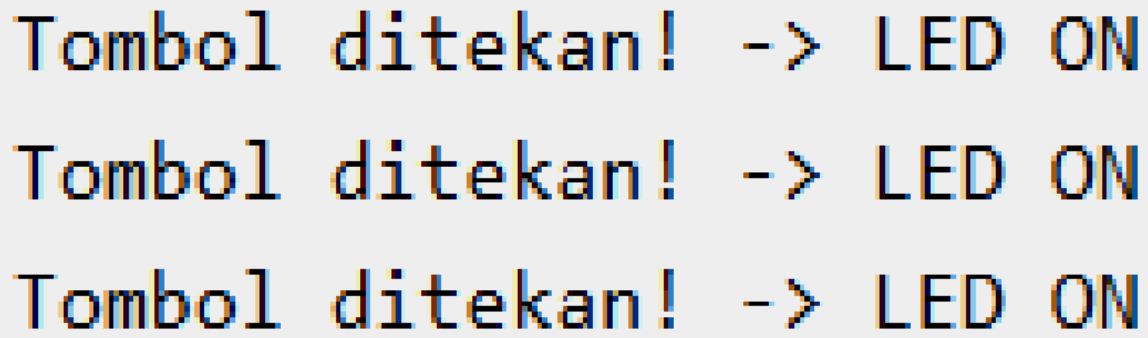
Gambar 8. Simulasi rangkaian Digital Input

- a. Push button berfungsi sebagai input yang memberikan perubahan kondisi ketika ditekan.
- b. Pin **D4/GPIO4** terhubung ke tombol (di antara push button dan resistor pull-down) untuk membaca status input tombol. Resistor pull-down $10k\Omega$ menjaga input tetap LOW ketika tombol tidak ditekan.
- c. Resistor pull-down $10k\Omega$ (terhubung ke GND via pin **D21/GPIO21**) menjaga input tetap LOW ketika tombol tidak ditekan. Saat tombol ditekan, input berubah menjadi HIGH sehingga program dapat meresponsnya.
- d. LED merah terhubung ke pin **D5/GPIO5** (dengan pembatas arus resistor 220Ω yang terhubung ke GND via pin **D23/GPIO23**) sebagai indikator hasil pembacaan tombol.
- e. Daya 3.3V diambil dari pin **3V3** menuju salah satu kaki push button, sehingga saat tombol ditekan, input pada pin D4 berubah menjadi HIGH.



Gambar 9. Kondisi rangkaian saat tombol ditekan

Saat tombol ditekan, arus listrik 3,3V mengalir ke pin D4/GPIO4 sehingga sistem membacanya sebagai sinyal HIGH. Kondisi ini memicu program untuk menyalakan LED sebagai respon visual. Ketika tombol dilepas, jalur 3,3V terputus dan resistor pull-down menarik pin D4 kembali ke posisi LOW (GND), yang menyebabkan LED otomatis padam.

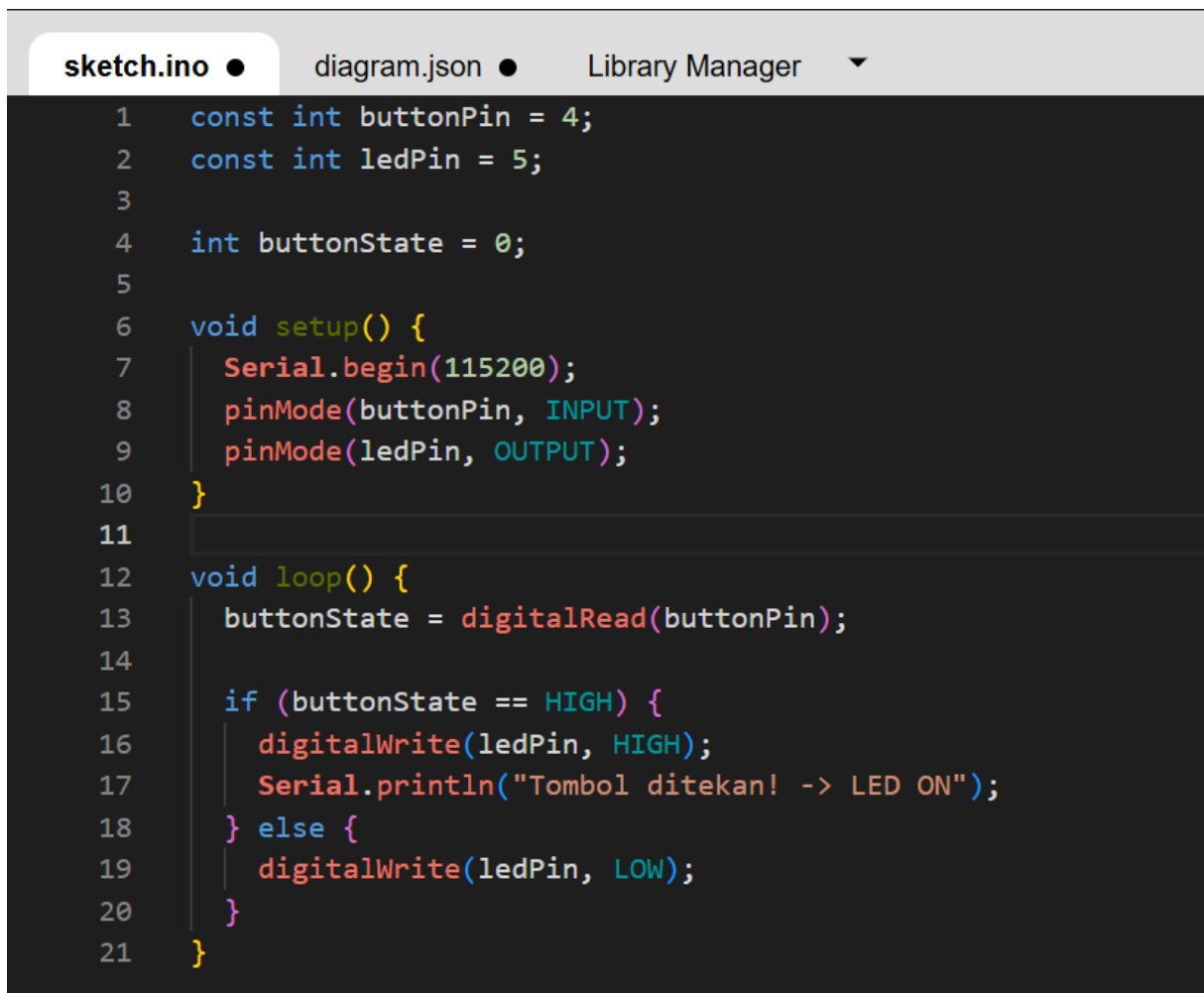


Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON
Tombol ditekan! -> LED ON

Gambar 10. Tampilan Serial Monitor saat tombol ditekan

Di Serial Monitor, muncul notifikasi kalau tombol lagi ditekan dan posisi LED dalam kondisi ON. Pesan ini bisa keluar gara-gara program berhasil ngebaca status input-nya pakai perintah `digitalRead()`.

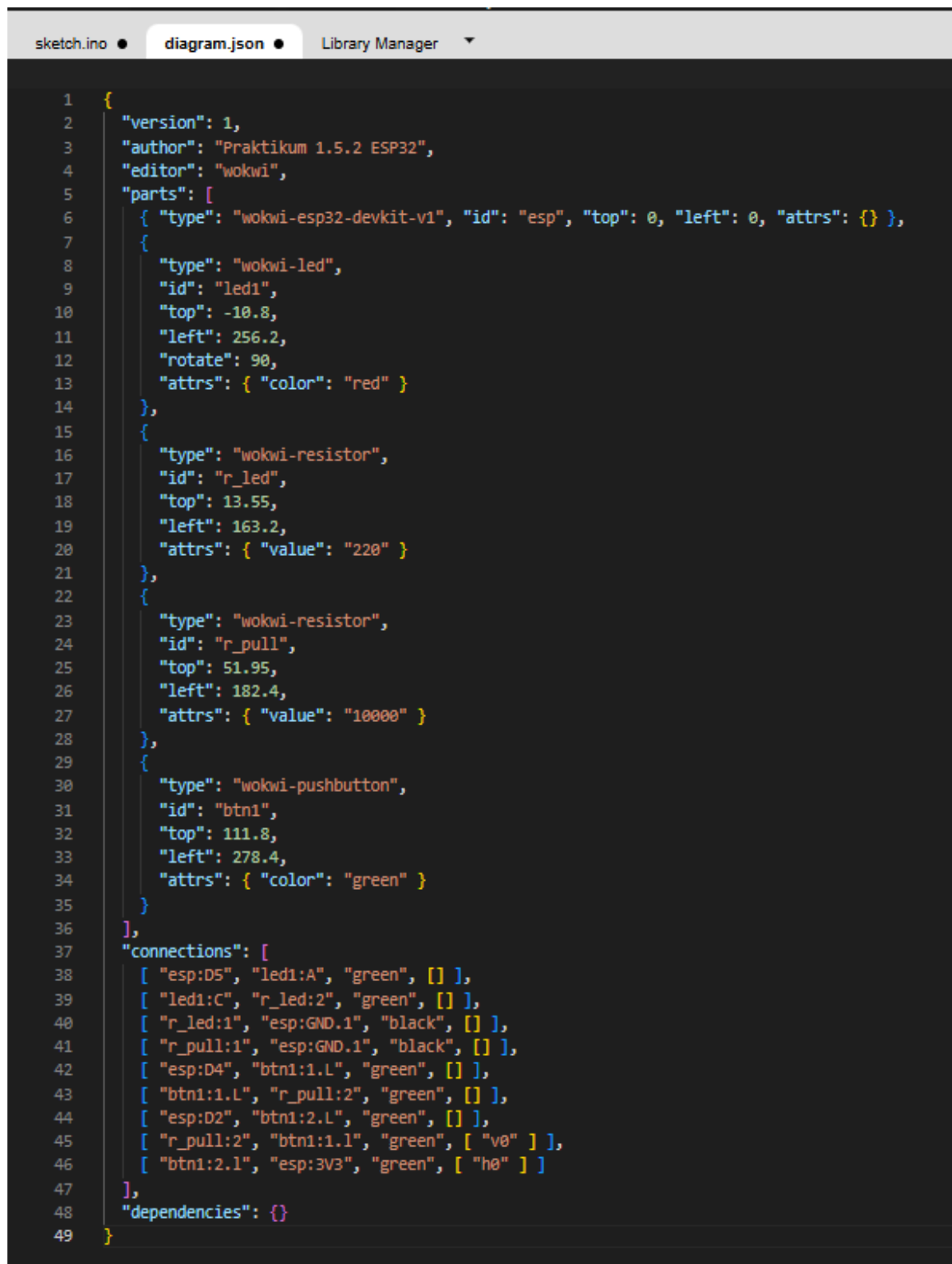
Karena di praktikum ini sifat tombolnya masih *momentary*, LED-nya cuma bakal nyala pas tombolnya ditahan aja. Begitu tombolnya dilepas, LED-nya otomatis mati lagi.



```
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  int buttonState = 0;
5
6  void setup() {
7    Serial.begin(115200);
8    pinMode(buttonPin, INPUT);
9    pinMode(ledPin, OUTPUT);
10 }
11
12 void loop() {
13   buttonState = digitalRead(buttonPin);
14
15   if (buttonState == HIGH) {
16     digitalWrite(ledPin, HIGH);
17     Serial.println("Tombol ditekan! -> LED ON");
18   } else {
19     digitalWrite(ledPin, LOW);
20   }
21 }
```

Gambar 11. Kode program sketch.ino Digital Input

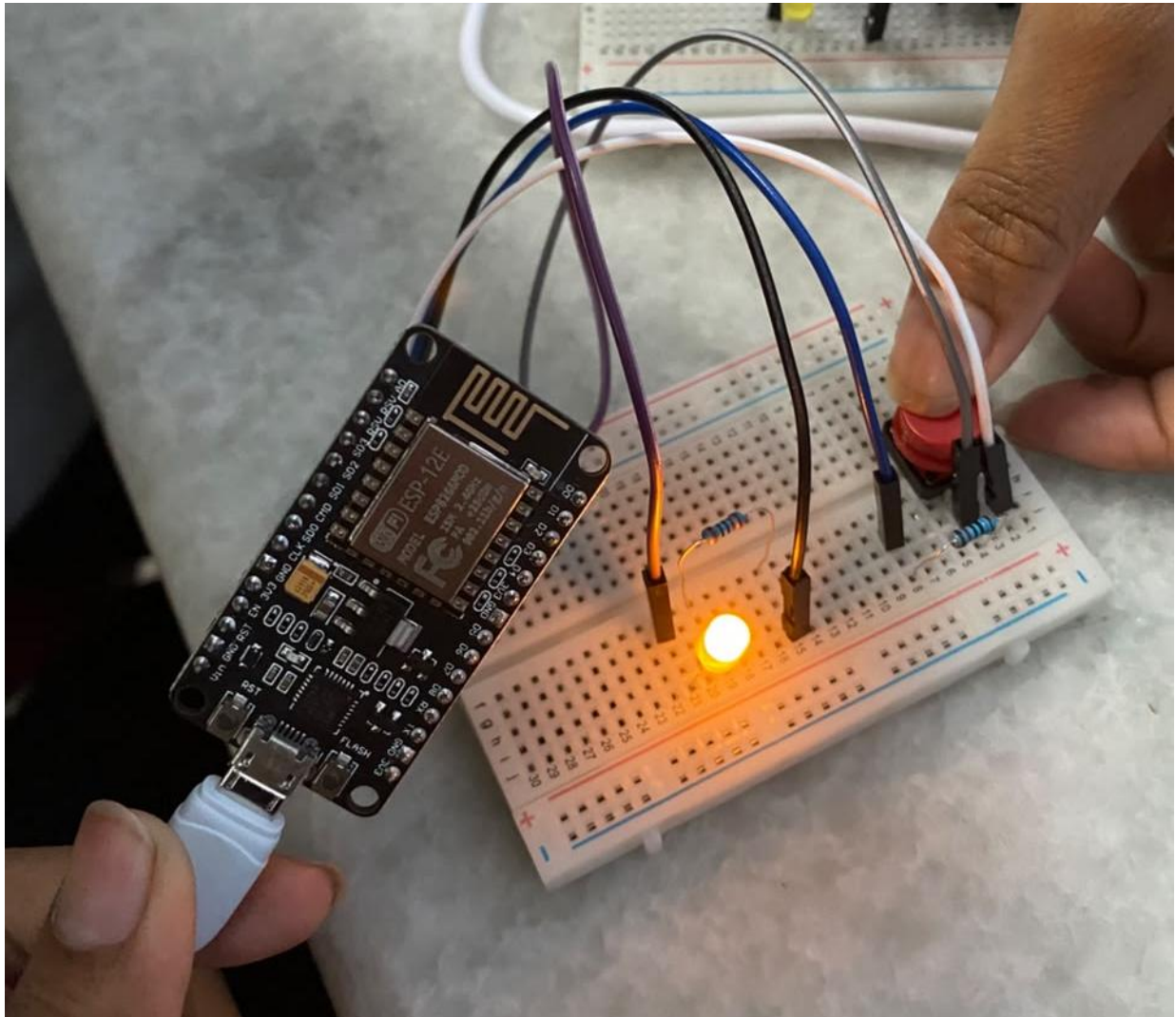
Kode program pada sketch.ino diawali dengan mengatur pin tombol sebagai input digital dan pin LED sebagai output. Melalui fungsi digitalRead(buttonPin), program memeriksa status tombol secara berkala. Ketika tombol terdeteksi bernilai HIGH, perintah digitalWrite(ledPin, HIGH) dieksekusi untuk menyalakan LED sekaligus mengirimkan notifikasi pada Serial Monitor. Jika tidak ditekan (LOW), LED langsung dimatikan melalui digitalWrite(ledPin, LOW).



```
1 {
2   "version": 1,
3   "author": "Praktikum 1.5.2 ESP32",
4   "editor": "wokwi",
5   "parts": [
6     { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": 0, "left": 0, "attrs": {} },
7     {
8       "type": "wokwi-led",
9       "id": "led1",
10      "top": -10.8,
11      "left": 256.2,
12      "rotate": 90,
13      "attrs": { "color": "red" }
14    },
15    {
16      "type": "wokwi-resistor",
17      "id": "r_led",
18      "top": 13.55,
19      "left": 163.2,
20      "attrs": { "value": "220" }
21    },
22    {
23      "type": "wokwi-resistor",
24      "id": "r_pull",
25      "top": 51.95,
26      "left": 182.4,
27      "attrs": { "value": "10000" }
28    },
29    {
30      "type": "wokwi-pushbutton",
31      "id": "btn1",
32      "top": 111.8,
33      "left": 278.4,
34      "attrs": { "color": "green" }
35    }
36  ],
37  "connections": [
38    [ "esp:D5", "led1:A", "green", [] ],
39    [ "led1:C", "r_led:2", "green", [] ],
40    [ "r_led:1", "esp:GND.1", "black", [] ],
41    [ "r_pull:1", "esp:GND.1", "black", [] ],
42    [ "esp:D4", "btn1:1.L", "green", [] ],
43    [ "btn1:1.L", "r_pull:2", "green", [] ],
44    [ "esp:D2", "btn1:2.L", "green", [] ],
45    [ "r_pull:2", "btn1:1.R", "green", [ "v0" ] ],
46    [ "btn1:2.R", "esp:3V3", "green", [ "h0" ] ]
47  ],
48  "dependencies": {}
49 }
```

Gambar 12. Bagian awal konfigurasi diagram.json

Diagram rangkaian terdiri dari board ESP32 (wokwi-esp32-devkit-v1), LED merah, resistor 220 ohm untuk LED, resistor 10k ohm untuk *pull-down*, *push button* hijau, dan kabel-kabel yang saling menghubungkan antar pin-nya. Semua komponen ini dirangkai sesuai konsep *input pull-down* yang dipakai di praktikum ini.



Gambar 13. Menggunakan Alat Secara Langsung

1.6 Latihan

Kerjakan latihan berikut untuk menguji pemahaman Anda. Jelaskan hasilnya di dalam laporan!

1. Latihan 1 (Pemahaman Logika Pin): Berdasarkan tabel Referensi Pinout di subbab 1.3.3, jelaskan apa yang mungkin terjadi jika Anda menggunakan logika rangkaian dari Praktikum 2 (Active-High / pull-down), namun menyambungkan tombol tersebut ke pin D8 (GPIO 15), lalu menekan tombol tersebut tepat pada detik di mana NodeMCU baru diberi daya?

Jawab: Pin D8 (GPIO 15) pada ESP8266/NodeMCU merupakan *strapping pin* yang menentukan *boot mode*. GPIO 15 dipersyaratkan harus bernilai LOW saat proses *booting* berlangsung agar ESP8266 dapat booting secara normal dari flash memory.

Dampak saat tombol ditekan tepat saat booting: Jika tombol berkonfigurasi *pull-down* (Active-High) ditekan saat daya baru diberikan, arus 3,3V akan mengalir ke pin D8 sehingga kondisinya menjadi HIGH pada detik pertama booting. Hal ini menyebabkan NodeMCU gagal masuk ke mode operasi normal (*normal boot failure* / mengalami *boot loop*) dan program tidak akan berjalan.

2. Latihan 2 (Implementasi Kode): Modifikasi kode pada Praktikum 1 agar LED berkedip secara tidak simetris, yaitu menyala sangat cepat selama 200 ms, kemudian mati agak lama selama 800 ms. Tuliskan dua baris fungsi `delay()` yang harus Anda ubah.

Jawab: `delay(200); // Durasi LED menyala cepat (200 ms)`
`delay(800); // Durasi LED mati lebih lama (800 ms)`

3. Latihan 3 (Analisis Sirkuit): Berdasarkan hukum aliran arus listrik searah, apa yang terjadi jika pada Praktikum 1 Anda memasang kaki LED secara terbalik (Anoda ke GND, Katoda ke D1)?

Apakah program akan mengalami error kompilasi?

Tidak. Sisi perangkat lunak/kode tetap valid dan berhasil dikompilasi karena error kompilasi (*syntax error*) hanya terjadi jika ada kesalahan penulisan tata bahasa kode.

Mengapa LED tidak menyala?

LED (*Light Emitting Diode*) adalah komponen dioda yang bersifat *polar* (memiliki arah polaritas) dan hanya mengalirkan arus secara searah (*forward bias*). Jika Anoda dihubungkan ke GND dan Katoda ke D1 (HIGH), LED berada dalam kondisi prabebas mundur (*reverse bias*) sehingga menahan aliran arus listrik dan LED tidak dapat menyala.

4. Latihan 4 (Pengembangan Algoritma): Ubah blok kondisi if-else pada Praktikum 2 sehingga perilakunya terbalik: "Saat tombol TIDAK ditekan, LED menyala. Saat tombol DITEKAN, LED justru mati."

Jawab:

```
if (buttonState == LOW) {  
    digitalWrite(ledPin, HIGH); // LED menyala saat tombol TIDAK ditekan  
    Serial.println("Tombol tidak ditekan -> LED ON");  
} else {  
    digitalWrite(ledPin, LOW); // LED mati saat tombol DITEKAN  
}
```

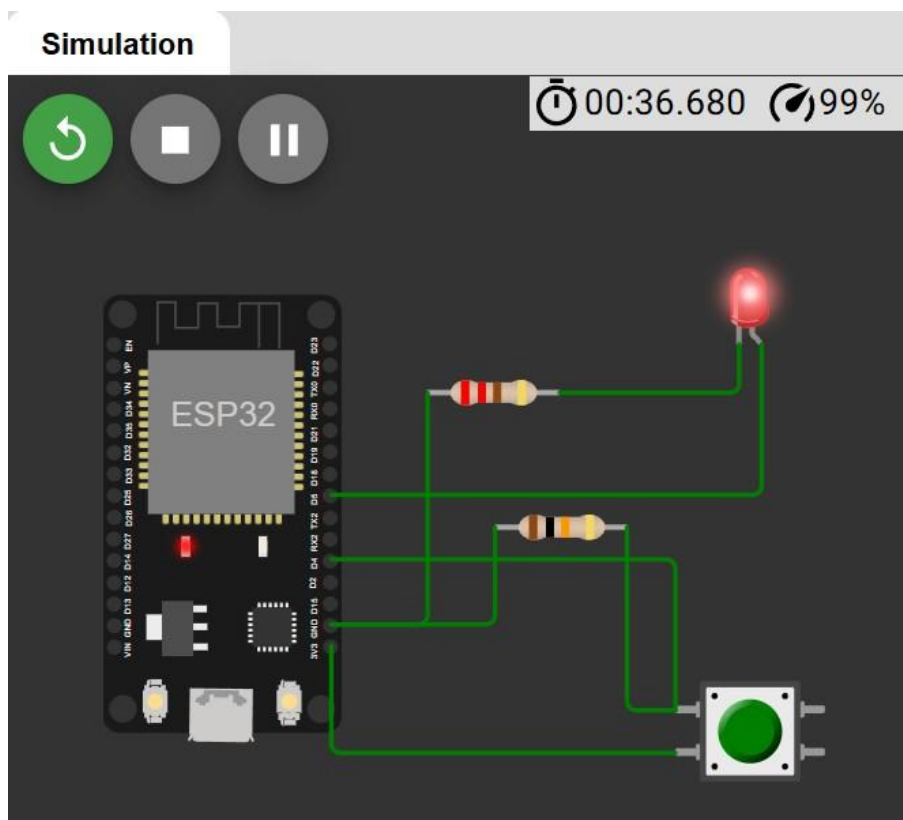
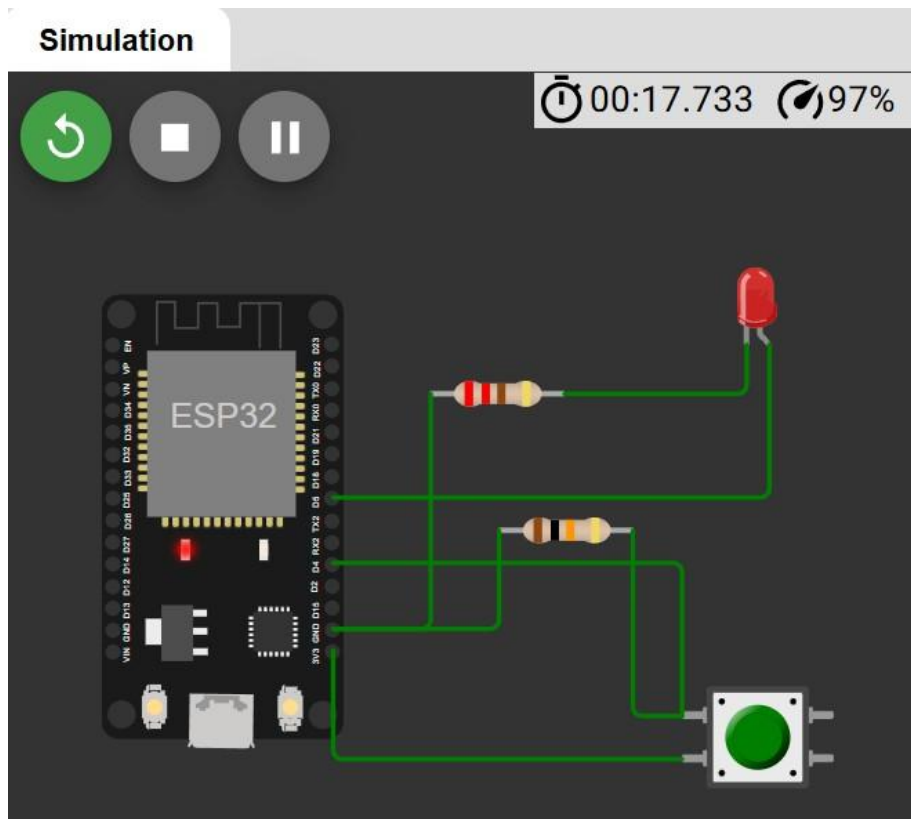
1.7 Tugas

Tugas ini harus dikerjakan secara mandiri untuk mengukur kemampuan integrasi dan logika pemrograman Anda.

Modifikasi Sistem Sakelar Toggle (Latching)

- Tujuan: Mampu mengimplementasikan logika pelacakan status (state tracking) dan membedakan antara perilaku tombol momentary dengan latching.
- Tingkat Kesulitan: Menengah
- Instruksi:
 - a. Gunakan konfigurasi hardware yang persis sama dengan Praktikum 2 (Rangkaian Pull-Down).
 - b. Modifikasi kode program sehingga tombol berfungsi layaknya sakelar lampu ruangan:
 - Saat sistem pertama dinyalakan, LED dalam keadaan mati.
 - Jika tombol ditekan 1 kali lalu dilepas, LED menyala dan tetap menyala.
 - Jika tombol ditekan 1 kali lagi lalu dilepas, LED mati dan tetap mati.
 - c. Petunjuk: Gunakan variabel penyimpanan status (misalnya tipe boolean atau int) untuk melacak apakah lampu saat ini sedang ON atau OFF. Anda juga membutuhkan instruksi `delay()` singkat (sekitar 200 milidetik) tepat setelah mendeteksi tombol ditekan, untuk mengatasi fenomena pantulan mekanis (debouncing pada tombol fisik).
- Kriteria Keberhasilan:
 - a. Perubahan status (ON ke OFF, atau sebaliknya) hanya terjadi dengan satu kali tekanan singkat (bukan ditahan).
 - b. Tidak ada fenomena flickering (lampu berkedip acak atau gagal mati) saat tombol ditekan.

- c. Program memiliki baris komentar yang menjelaskan bagaimana logika pelacakan state dan debounce bekerja.



Gambar 14. Kondisi rangkaian saat tombol ditekan

Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF
Tombol ditekan -> LED ON
Tombol ditekan -> LED OFF



```
1  const int ledPin = 5;      // LED terhubung ke GPIO 23
2  const int buttonPin = 4;   // Push button terhubung ke GPIO 22
3
4  bool ledState = false;
5
6  void setup() {
7      Serial.begin(115200);
8
9      pinMode(ledPin, OUTPUT);
10     pinMode(buttonPin, INPUT);
11
12     // Kondisi awal LED mati
13     digitalWrite(ledPin, LOW);
14
15     Serial.println("=== SISTEM SAKELAR TOGGLE ESP32 ===");
16     Serial.println("LED awal: OFF");
17 }
18
19 void loop() {
20     int buttonState = digitalRead(buttonPin);
21
22     // Jika tombol ditekan
23     if (buttonState == HIGH) {
24
25         // Mengubah kondisi LED
26         ledState = !ledState;
27
28         digitalWrite(ledPin, ledState);
29
30         if (ledState) {
31             Serial.println("Tombol ditekan -> LED ON");
32         } else {
33             Serial.println("Tombol ditekan -> LED OFF");
34         }
35
36         // Debounce untuk mencegah satu tekanan
37         // terbaca berkali-kali
38         delay(200);
39
40         // Menunggu tombol dilepas
41         while (digitalRead(buttonPin) == HIGH) {
42             delay(10);
43         }
44     }
45 }
```

Gambar 15. Kode program sketch.ino Digital Input

sketch.ino

diagram.json

Library Manager

```

1  {
2    "version": 1,
3    "author": "Tugas 1.7 ESP32 - Toggle Latching Switch",
4    "editor": "wokwi",
5    "parts": [
6      { "type": "wokwi-esp32-devkit-v1", "id": "esp", "top": -120.1, "left": -24.2, "attrs": {} },
7      { "type": "wokwi-led", "id": "led1", "top": -138, "left": 243.8, "attrs": { "color": "red" } },
8      {
9        "type": "wokwi-resistor",
10       "id": "r_led",
11       "top": -80,
12       "left": 120,
13       "attrs": { "value": "220" }
14     },
15     {
16       "type": "wokwi-resistor",
17       "id": "r_pull",
18       "top": -20,
19       "left": 150,
20       "attrs": { "value": "10000" }
21     },
22     {
23       "type": "wokwi-pushbutton",
24       "id": "btn1",
25       "top": 54.2,
26       "left": 230.4,
27       "attrs": { "color": "green" }
28     }
29   ],
30   "connections": [
31     [ "esp:D5", "led1:A", "green", [ "h0" ] ],
32     [ "led1:C", "r_led:2", "green", [ "v0" ] ],
33     [ "r_led:1", "esp:GND.1", "green", [ "v0" ] ],
34     [ "r_pull:1", "esp:GND.1", "green", [ "v0" ] ],
35     [ "esp:D4", "btn1:1.1", "green", [ "h0" ] ],
36     [ "btn1:1.1", "r_pull:2", "green", [ "h0" ] ],
37     [ "btn1:2.1", "esp:3V3", "green", [ "h0" ] ]
38   ],
39   "dependencies": {}
40 }

```

Gambar 16. Bagian awal konfigurasi diagram.json

Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

SKETCHBOOK

Blink

praktikum_gambar_1_sketch

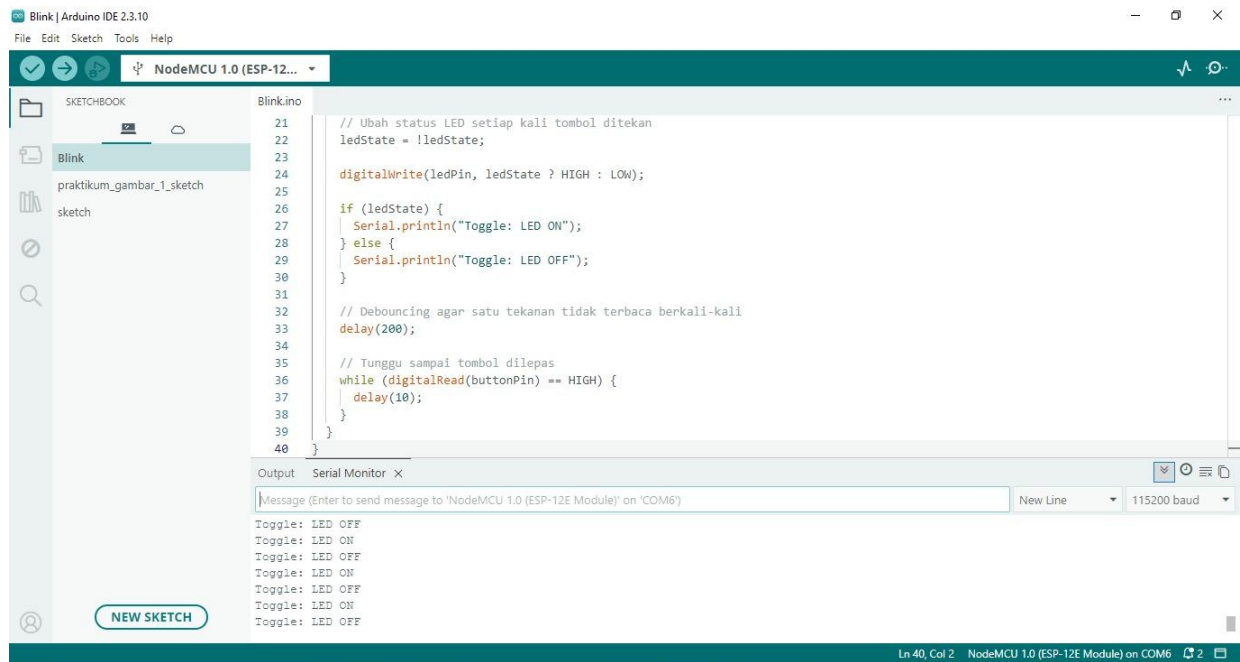
sketch

Blink.ino

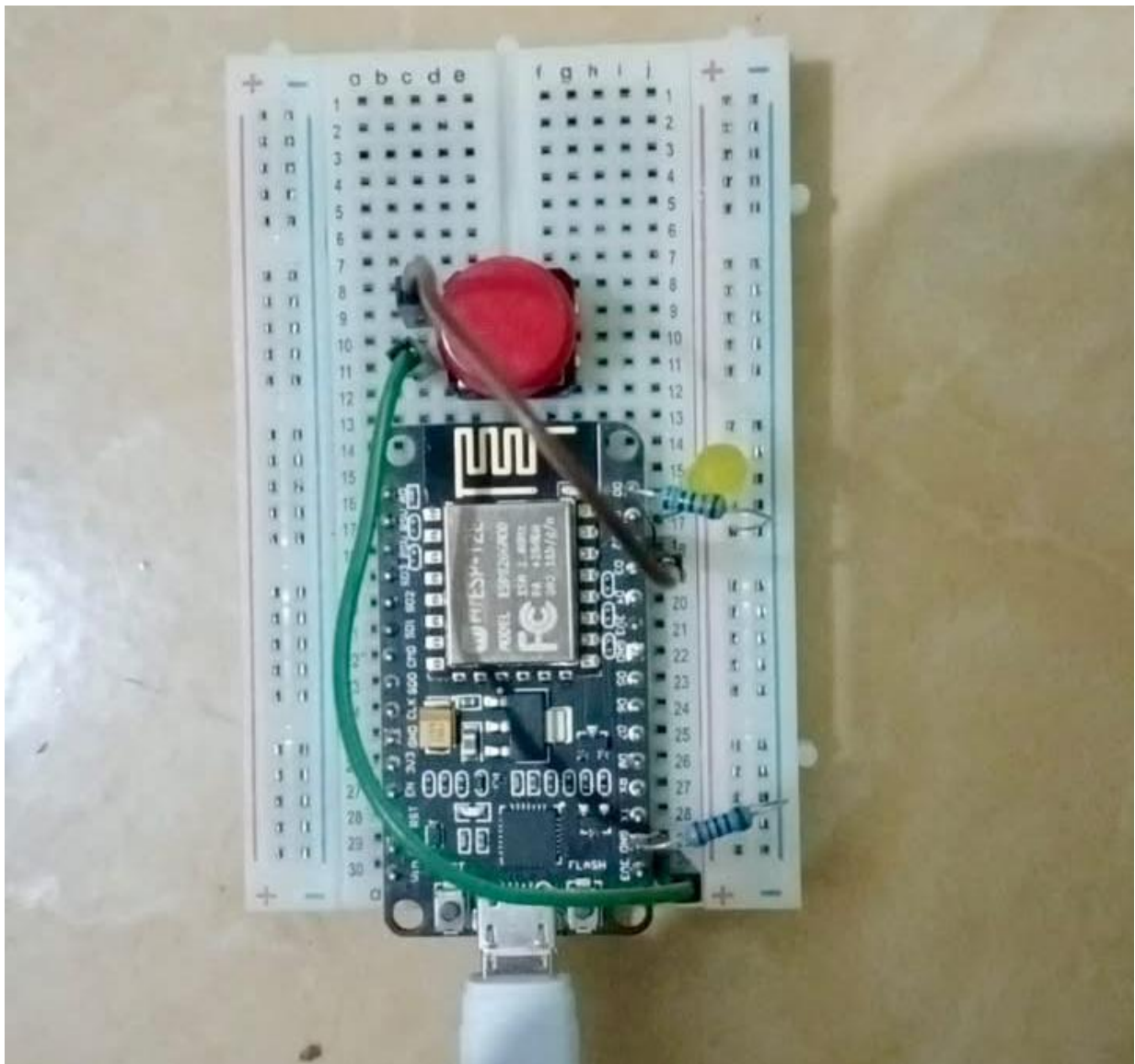
```

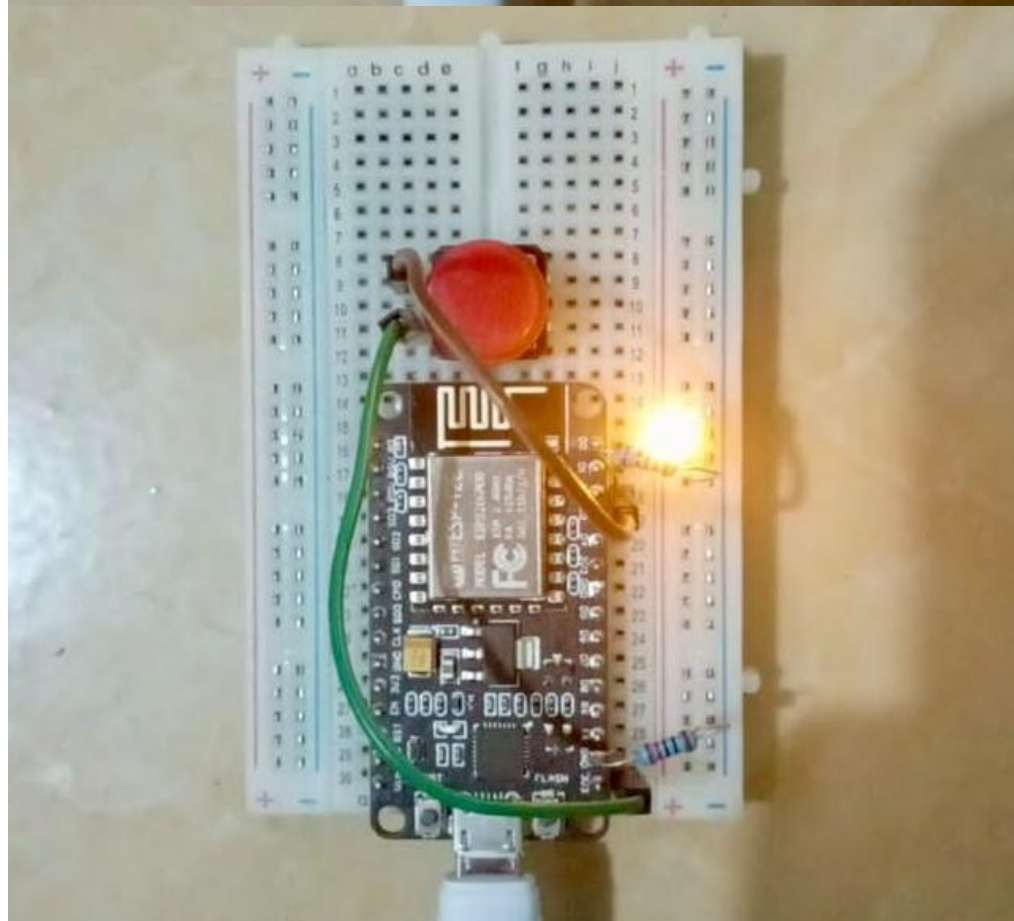
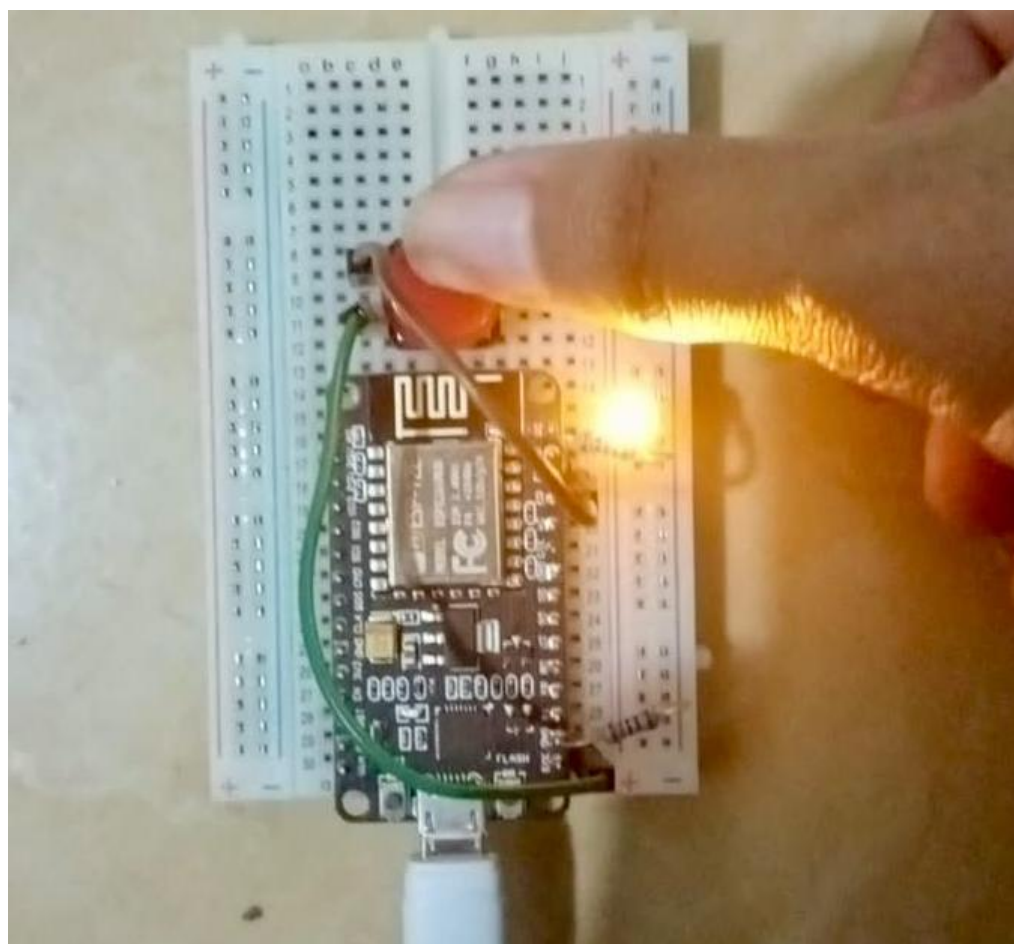
1  const int buttonPin = 4;
2  const int ledPin = 5;
3
4  bool ledState = false; // Menyimpan status LED: false = mati
5  int buttonState = LOW;
6
7  void setup() {
8    Serial.begin(115200);
9
10   pinMode(buttonPin, INPUT);
11   pinMode(ledPin, OUTPUT);
12
13   // Saat awal dinyalakan, LED dalam keadaan mati
14   digitalWrite(ledPin, LOW);
15 }
16
17 void loop() {
18   buttonState = digitalRead(buttonPin);
19
20   if (buttonState == HIGH) {

```



Gambar 17. Simulasi Menggunakan Arduino





2.4 Persiapan Praktikum

2.4.1 Kebutuhan Perangkat Keras

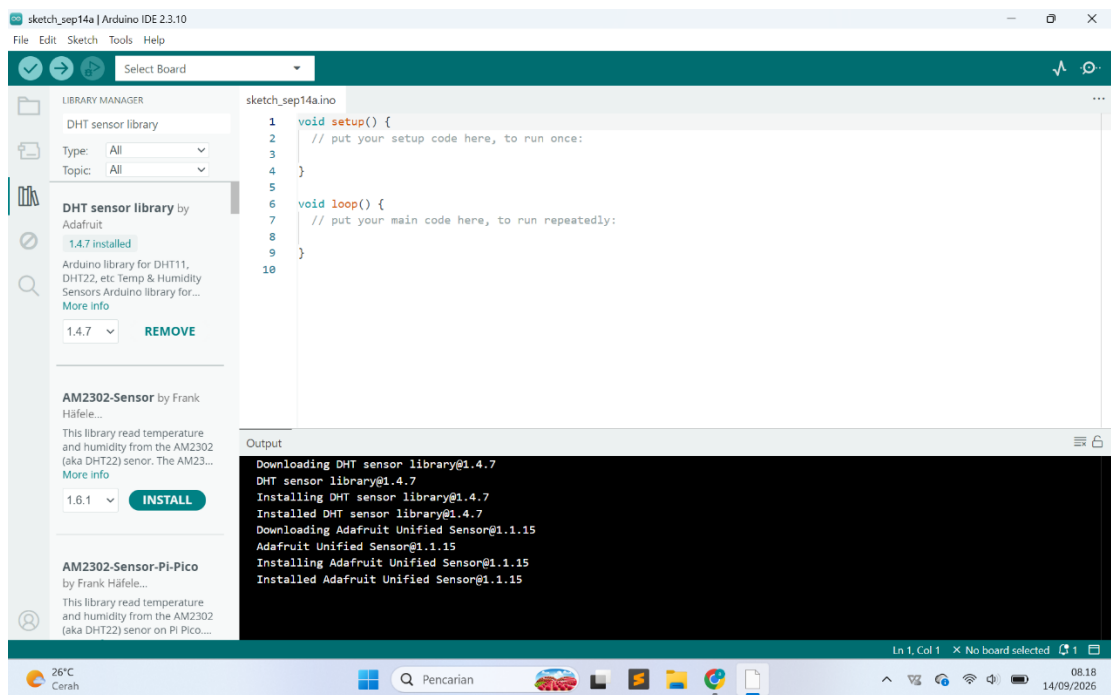
Pastikan komponen berikut tersedia di meja Anda:

- *Board* NodeMCU ESP8266 (1 buah)
- Kabel Micro USB (1 buah)
- *Breadboard* (1 buah)
- Sensor LDR (1 buah)
- Resistor 10k Ohm (1 buah, untuk pembagi tegangan LDR)
- Sensor Suhu & Kelembapan (DHT11 atau DHT22) (1 buah)
- Modul Relay 1-Channel 5V (1 buah)
- LED 5mm (1 buah)
- Resistor 220 Ohm (1 buah, untuk pembatas arus LED)
- Kabel *Jumper Male-to-Male* dan *Male-to-Female* (secukupnya)

2.4.2 Persiapan Perangkat Lunak (Instalasi *Library* DHT)

Membaca sinyal digital *single-bus* dari sensor DHT sangat rumit jika dilakukan manual karena membutuhkan presisi waktu dalam hitungan *microsecond*. Oleh karena itu, kita wajib menginstal perangkat lunak tambahan berupa *Library*.

1. Pada Arduino IDE, buka menu **Sketch > Include Library > Manage Libraries...**
2. Ketikkan kata kunci "**DHT sensor library**" di kotak pencarian.
3. Cari pustaka yang dibuat secara resmi oleh **Adafruit** dan klik **Install**.



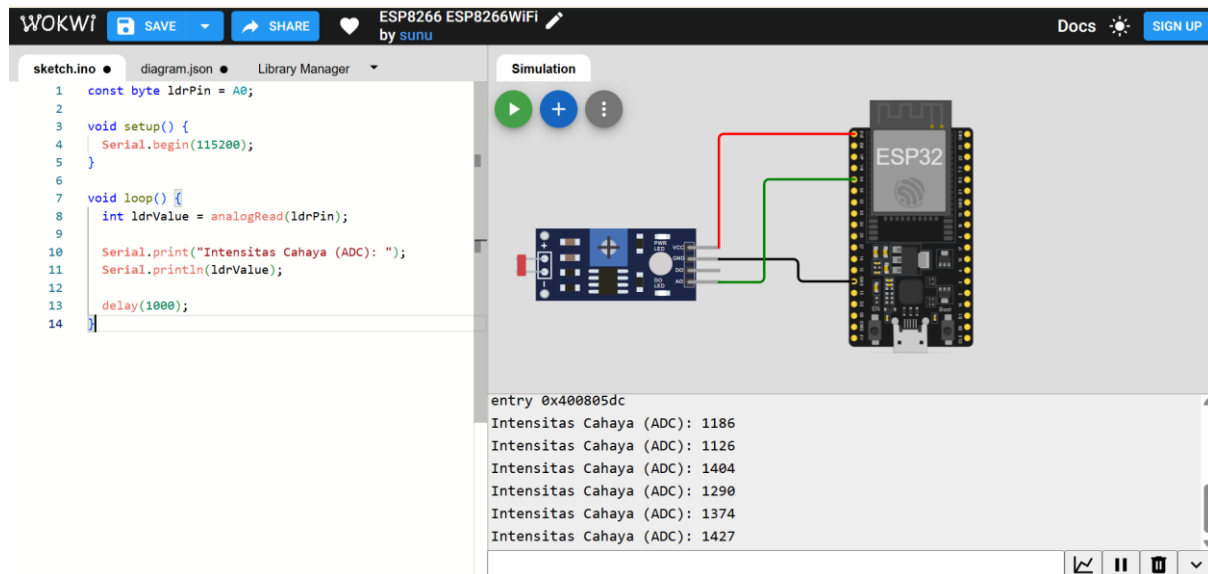
4. *Penting*) Jika muncul dialog persetujuan (*prompts*) untuk menginstal pustaka dependen/tambahan, pastikan Anda menekan tombol "**Install All**". (Library ini membutuhkan *Adafruit Unified Sensor* agar dapat beroperasi).

2.5 Praktikum

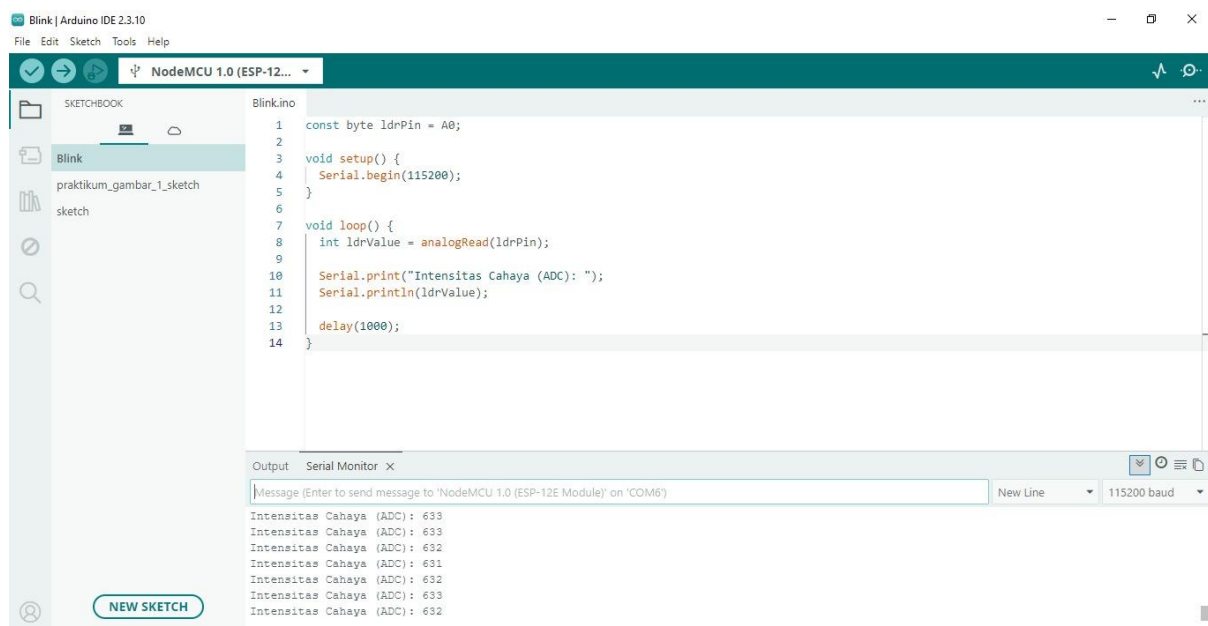
2.5.1 Praktikum 1: Pembacaan Intensitas Cahaya (LDR) via ADC

Sensor LDR akan menurunkan hambatannya (resistansi) saat terpapar cahaya terang, dan meningkatkan

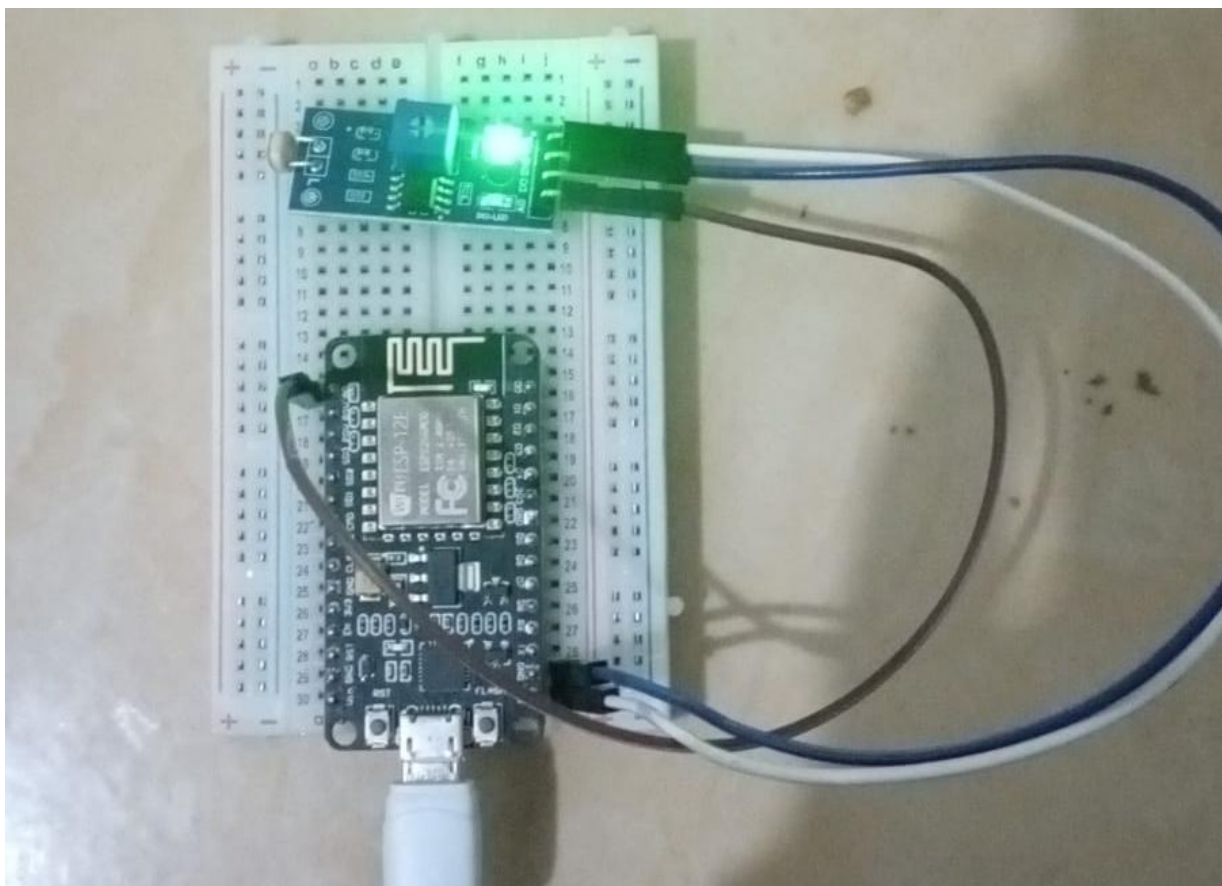
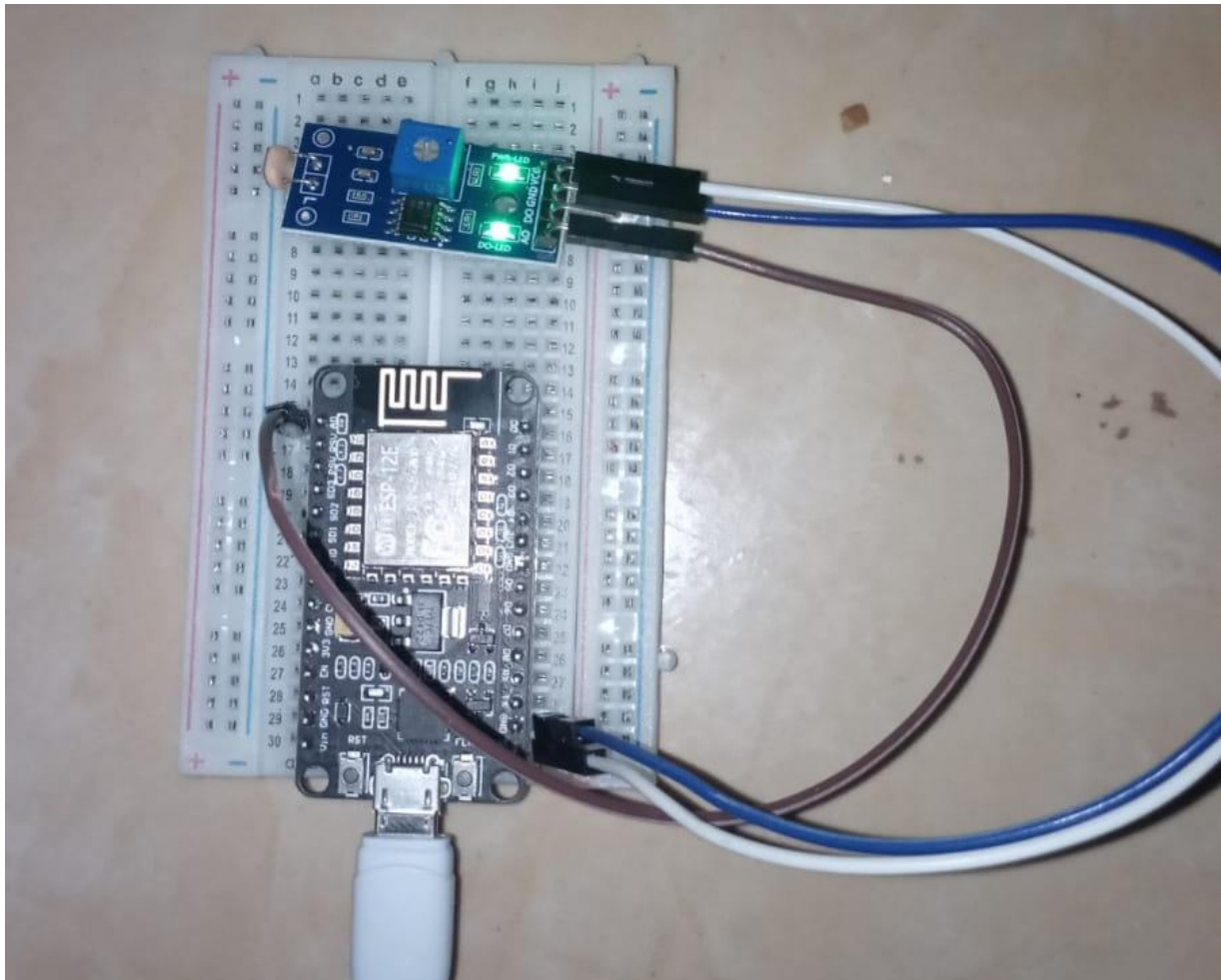
resistansinya saat gelap. Untuk menerjemahkan perubahan resistansi ini menjadi variasi tegangan yang bisa diukur oleh pin A0, kita menggunakan sirkuit pembagi tegangan (*Voltage Divider*).



Gambar 2. Simulasi Menggunakan Wokwi



Gambar 3. Simulasi Menggunakan Arduino



Gambar 4. Menggunakan Alat Secara Langsung

Hasil Implementasi Sensor LDR pada ESP32

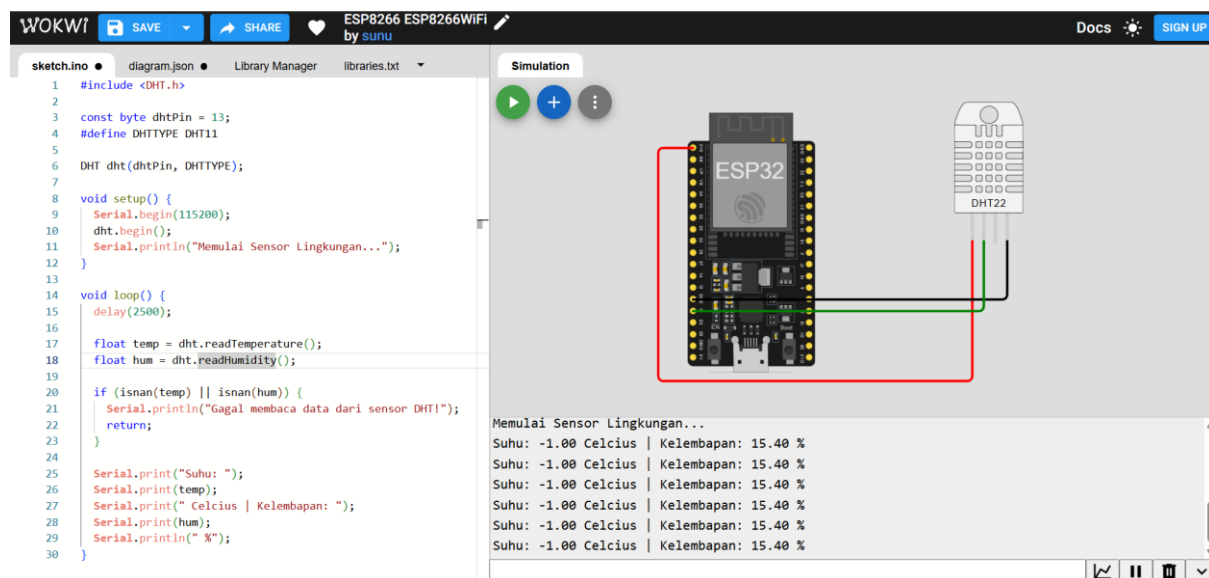
Pada percobaan ini dilakukan simulasi pembacaan intensitas cahaya menggunakan **sensor** LDR (Light Dependent Resistor) yang dihubungkan dengan mikrokontroler ESP32 pada Wokwi. Sensor LDR digunakan untuk mendeteksi perubahan intensitas cahaya berdasarkan nilai ADC yang dibaca oleh ESP32. Pin analog sensor dihubungkan ke salah satu pin ADC ESP32, sedangkan sensor juga dihubungkan ke sumber tegangan dan GND.

Program menggunakan `analogRead()` untuk membaca nilai analog dari sensor LDR. Nilai hasil pembacaan kemudian ditampilkan melalui Serial Monitor dengan kecepatan komunikasi 115200 baud. Pembacaan dilakukan setiap satu detik sehingga nilai intensitas cahaya dapat diamati secara berkala.

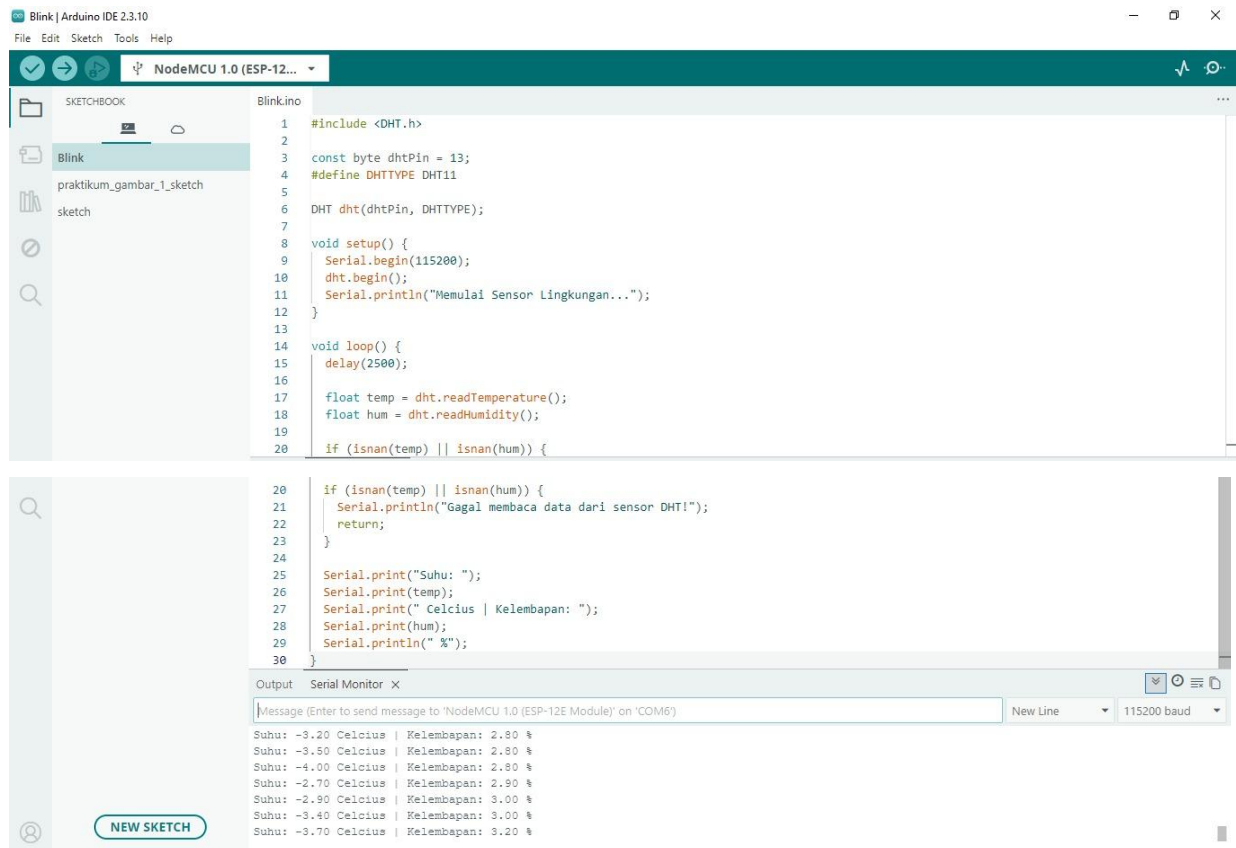
Berdasarkan hasil simulasi pada gambar, diperoleh beberapa nilai ADC, yaitu 1186, 1126, 1404, 1290, 1374, dan 1427. Nilai tersebut menunjukkan bahwa sensor LDR berhasil membaca perubahan intensitas cahaya dan mengirimkan hasil pembacaannya ke ESP32. Semakin besar perubahan cahaya yang diterima sensor, maka nilai ADC yang terbaca juga dapat mengalami perubahan.

2.5.2 Praktikum 2: Membaca Data Sensor Suhu & Kelembapan (DHT)

Sensor DHT menggunakan protokol komunikasi digital *single-wire*. Artinya, seluruh data (suhu dan kelembapan) ditransmisikan melalui satu kabel data saja. Protokol ini sangat bergantung pada waktu (*timing*). Mikrokontroler harus mengirimkan perintah "minta data" dengan durasi *HIGH* dan *LOW* yang sangat presisi. Setelah itu, sensor akan mengirimkan kembali serangkaian pulsa digital yang panjangnya spesifik menunjukkan nilai suhu dan kelembapan.



Gambar 5. Simulasi Menggunakan Woktwi



Gambar 6. Simulasi Menggunakan Arduino



Gambar 7. Menggunakan Alat Secara Langsung

Hasil Implementasi Sensor DHT22 pada ESP32

Pada percobaan ini dilakukan simulasi pembacaan kondisi lingkungan menggunakan sensor DHT22

yang dihubungkan dengan mikrokontroler ESP32 pada Wokwi. Sensor DHT22 digunakan untuk mengukur dua parameter lingkungan, yaitu suhu dalam satuan Celsius dan kelembapan dalam satuan persen (%). Sensor dihubungkan ke ESP32 menggunakan pin digital, serta mendapatkan sumber tegangan dan GND.

Program menggunakan library DHT.h untuk mengakses sensor DHT22. Pada program, pin data sensor diatur menggunakan GPIO 13. ESP32 kemudian membaca nilai suhu dan kelembapan setiap 2,5 detik dan menampilkannya pada Serial Monitor dengan baud rate 115200.

Berdasarkan hasil simulasi pada gambar, Serial Monitor menampilkan hasil pembacaan berupa suhu -1,00 Celsius dan kelembapan 15,40% secara berulang. Hal ini menunjukkan bahwa ESP32 berhasil menjalankan program dan menerima data dari sensor DHT22. Nilai yang ditampilkan dapat digunakan untuk mengetahui kondisi lingkungan yang terukur oleh sensor.

2.6 Latihan

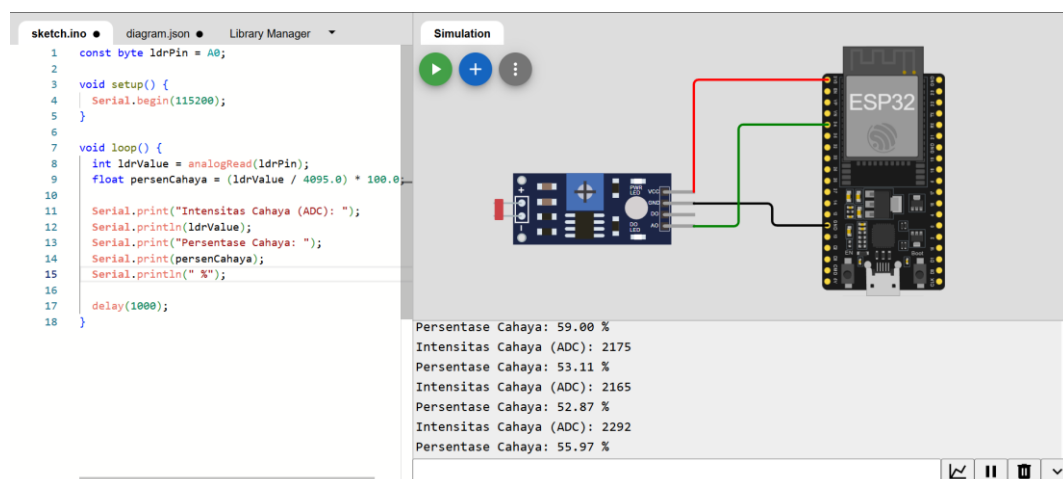
Kerjakan latihan berikut untuk menguji retensi konsep Anda secara mandiri.

- **Latihan 1 (Pemahaman Sistem ADC):** Mengapa batas maksimal nilai intensitas cahaya yang terbaca pada Serial Monitor di Praktikum 1 adalah sekitar angka 1023 (bukan 100 atau 1000)? Kaitkan jawaban Anda dengan penjelasan "Resolusi ADC 10-bit".

Jawab: Nilai maksimal sekitar **1023** terjadi karena ADC yang digunakan memiliki resolusi 10-bit. ADC 10-bit memiliki $2^{10} = 1024$ kemungkinan nilai, yaitu mulai dari 0 sampai 1023. Jadi, ketika tegangan analog yang masuk mencapai nilai maksimum, hasil pembacaan ADC adalah sekitar 1023. Dengan kata lain, angka 1023 bukan menunjukkan persentase cahaya, tetapi merupakan nilai hasil konversi sinyal analog menjadi data digital. Semakin besar nilai ADC, semakin besar nilai tegangan yang terbaca oleh ADC.

Pada percobaan saya kali ini yang menggunakan ESP32, hasil ADC dapat mencapai sekitar 4095 karena ESP32 menggunakan ADC dengan resolusi hingga 12-bit. Jadi angka 0–1023 pada latihan ini mengacu pada konteks ADC 10-bit yang dijelaskan di soal.

- **Latihan 2 (Implementasi Algoritma):** Nilai ADC 0-1023 tidak lazim dibaca oleh pengguna akhir (manusia). Modifikasi kode pada Praktikum 1 agar program menghitung dan mencetak persentase intensitas cahaya (rentang 0% hingga 100%). Tuliskan rumus matematika / perintah pembagian yang Anda sematkan di kode tersebut!



Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

SKETCHBOOK

Blink

praktikum_gambar_1_sketch

sketch

```
1 const byte ldrPin = A0;
2
3 void setup() {
4   Serial.begin(115200);
5 }
6
7 void loop() {
8   int ldrValue = analogRead(ldrPin);
9
10  Serial.print("Intensitas Cahaya (ADC): ");
11  Serial.println(ldrValue);
12
13  delay(1000);
14 }
```

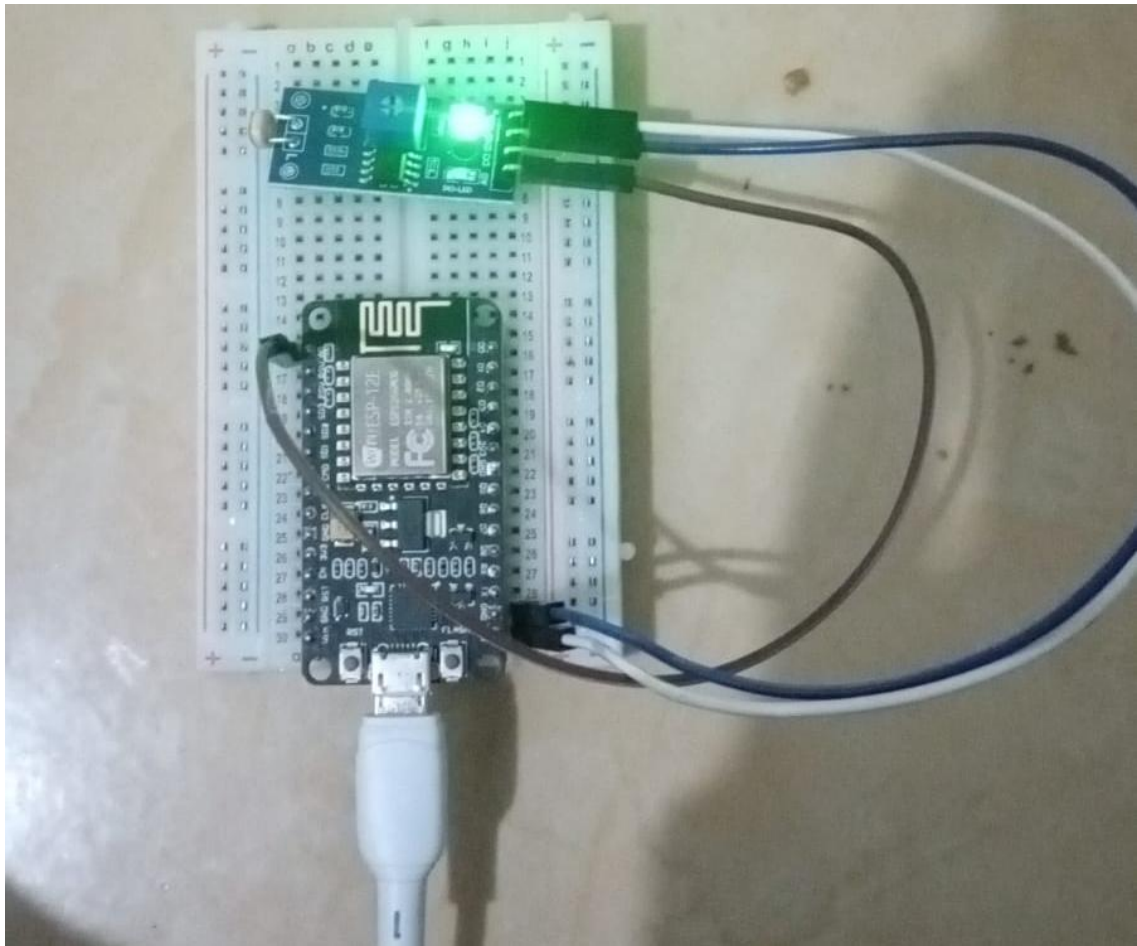
Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

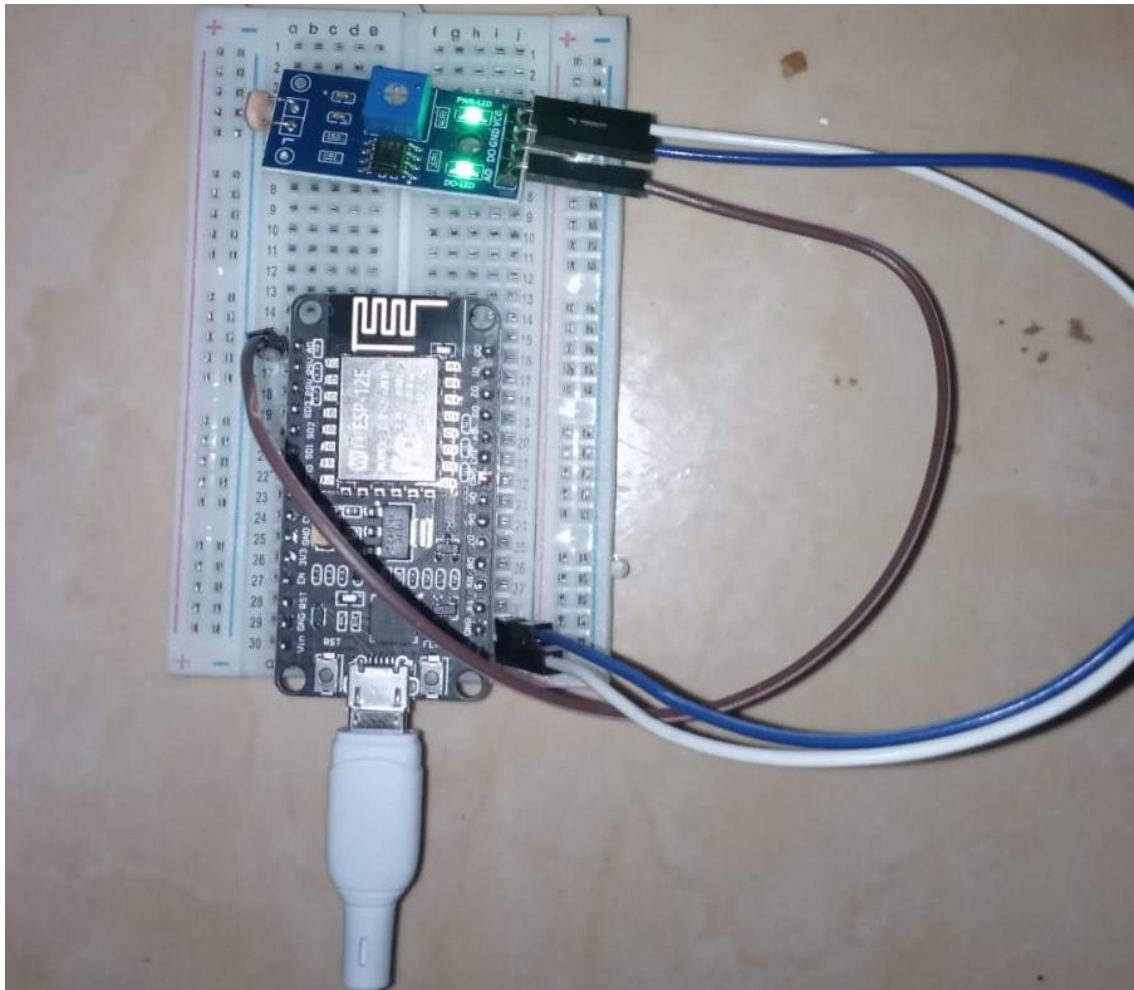
New Line 115200 baud

Intensitas Cahaya (ADC): 633
Intensitas Cahaya (ADC): 633
Intensitas Cahaya (ADC): 632
Intensitas Cahaya (ADC): 631
Intensitas Cahaya (ADC): 632
Intensitas Cahaya (ADC): 633
Intensitas Cahaya (ADC): 632

NEW SKETCH



Tidak terkena cahaya hidup 1



Jika terkena cahaya

Jawab: Nilai ADC yang diperoleh dari sensor LDR diubah menjadi persentase agar lebih mudah dipahami oleh pengguna. Karena mikrokontroler yang digunakan adalah ESP32, rentang nilai ADC yang digunakan adalah 0–4095. Persentase intensitas cahaya dihitung dengan membagi nilai ADC dengan nilai maksimum 4095, kemudian dikalikan 100.

Rumus yang digunakan:

$$\text{Persentase Cahaya} = \frac{\text{Nilai ADC}}{4095} \times 100\%$$

Pada program, rumus tersebut dituliskan sebagai:

```
float persentase = (ldrValue / 4095.0) * 100.0;
```

Berdasarkan hasil simulasi pada Serial Monitor, terlihat nilai ADC seperti 2175, 2165, dan 2292 yang kemudian dikonversi menjadi persentase intensitas cahaya sekitar 53,11%, 52,87%, dan 55,97%.

- **Latihan 3 (Analisis Rangkaian):** Di subbab 2.3.4 disebutkan sifat modul *Relay* yang terisolasi *Optocoupler*. Banyak *Relay* di pasaran yang bekerja secara **Active-Low**. Jika *Relay* Anda bertipe *Active-Low*, apa yang terjadi pada aktuator (misal: lampu ruangan) yang terhubung ke *Relay* jika program Anda mengeksekusi `digitalWrite(relayPin, HIGH)`?

Jawab: Jika *Relay* yang digunakan bertipe *Active-Low*, maka logikanya berlawanan dengan

Relay Active-High.

Pada Relay Active-Low:

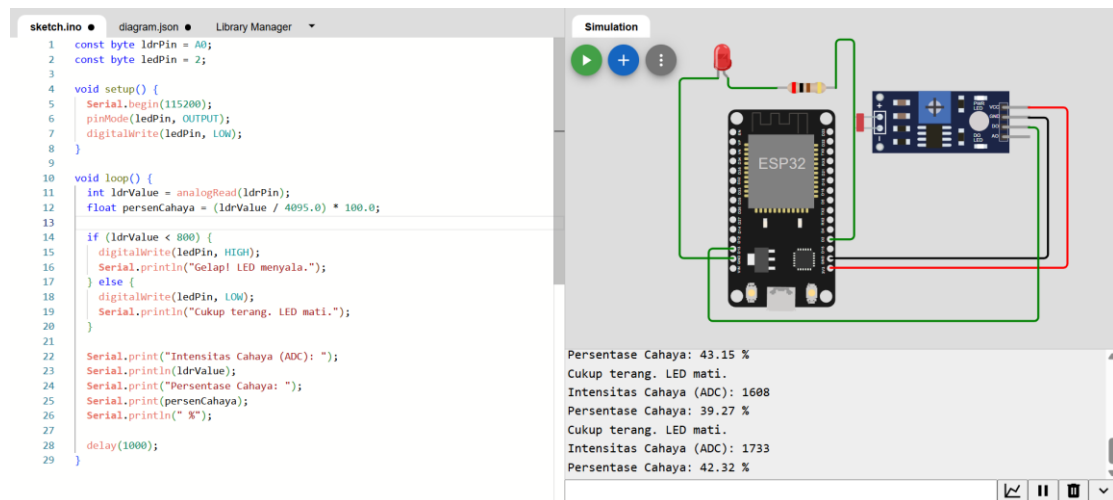
- LOW → Relay aktif/menyalakan
- HIGH → Relay tidak aktif/mati

Jadi, kalo program menjalankan:

`digitalWrite(relayPin, HIGH);`

maka Relay akan mati atau tidak aktif. Akibatnya, aktuator seperti lampu yang dikendalikan oleh Relay juga akan berada pada kondisi mati, tergantung konfigurasi kontak NO/NC pada Relay.

- **Latihan 4 (Pengembangan Logika):** Gabungkan wawasan dari Praktikum 1 Modul 2 ini dengan logika kendali LED dari Praktikum Modul 1. Gunakan blok kondisional if yang berfungsi menyalakan sebuah LED (D1) **HANYA JIKA** sensor LDR mendeteksi lingkungan yang gelap gulita (misalnya: ambang batas ADC berada di bawah angka 200).



Jawab: Sensor LDR digunakan untuk mengendalikan LED berdasarkan intensitas cahaya yang terbaca. Program menggunakan kondisi if dengan ambang batas ADC 800. Jika nilai ADC LDR kurang dari 800, maka LED akan menyala. Jika nilai ADC 800 atau lebih, maka LED akan mati.

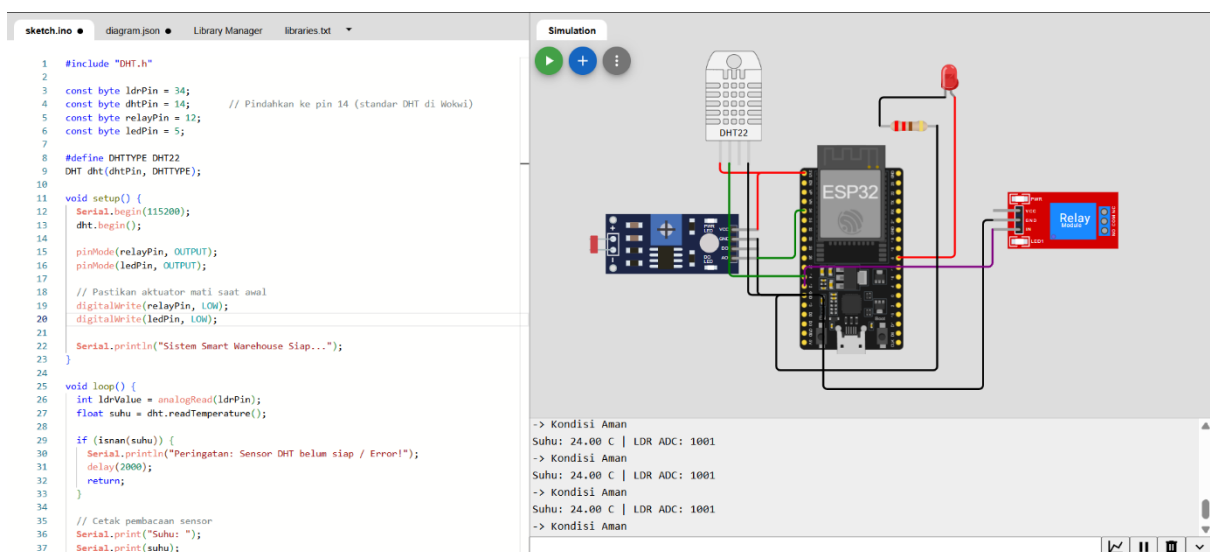
Berdasarkan hasil simulasi pada screenshot saya di atas, nilai ADC yang terbaca adalah 1608 dan 1733. Karena kedua nilai tersebut lebih besar dari 800, maka kondisi dianggap cukup terang dan LED mati.

2.7 Tugas: Sistem Otomasi Kontrol Lokal Terpadu

Sebuah sistem *Internet of Things (IoT)* harus memiliki kecerdasan dasar pada perangkat lapangannya (*Edge Computing*), sehingga apabila sewaktu-waktu koneksi internet terputus, perangkat masih dapat mengambil keputusan darurat berdasarkan kondisi lingkungan lokalnya.

Skenario Proyek (Smart Warehouse): Anda ditugaskan merancang *rule engine* aktuator mandiri. Sistem harus membaca Suhu (DHT) dan Intensitas Cahaya (LDR) secara bersamaan. Modul **Relay 5V** akan disimulasikan sebagai sakelar daya untuk pendingin ruangan (*Exhaust Fan*) atau sistem pencahayaan darurat.

- **Tujuan:** Mampu memadukan akuisisi multi-sensor (Analog LDR dan Digital DHT) dan menggunakan aktuator Relay untuk mengeksekusi keputusan terpadu berbasis kondisi/ambang batas.
- **Tingkat Kesulitan:** Menengah-Lanjut
- **Instruksi Perangkaian:**
 - Pertahankan sirkuit sensor LDR (di A0) dan DHT (di D7).
 - Hubungkan pin **VCC / +5V** dari Modul Relay ke pin **VIN** atau **VU** NodeMCU (mengambil daya murni 5V dari USB). **Jangan menyambungkan modul Relay 5V ke pin 3V3!**
 - Hubungkan pin **GND** Relay ke jalur **GND** bersama NodeMCU.
 - Hubungkan pin **IN / Signal** Relay ke pin **D6 (GPIO 12)**.
 - (Opsional sebagai indikator visual tambahan):* Pasang sebuah LED beresistor 220 Ohm pada pin **D1 (GPIO 5)**.
- **Instruksi Pemrograman (Aturan/Rule Engine):**
 - Konfigurasi pin Relay (D6) dan LED (D1) sebagai OUTPUT. (Kirimkan perintah mati di awal fungsi setup() untuk keamanan).
 - Buatlah struktur logika perbandingan (if ... else) menggunakan operator logika **OR (||)**.
 - Aktifkan Relay dan nyalakan LED indikator JIKA** kondisi lingkungan buruk berikut terpenuhi:
 - Suhu udara terbaca **di atas 34 Celcius** (Terlalu panas).
 - ATAU Nilai ADC LDR terbaca **di bawah angka 300** (Terlalu gelap).
 - Jika suhu normal dan ruangan cukup cahaya, pastikan Relay dan LED kembali mati.
 - Cetak status sistem ke Serial Monitor secara rutin (Contoh: "Kondisi Aman" / "Peringatan: Aktuator Aktif!").



Blink | Arduino IDE 2.3.10

File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12E...)

SKETCHBOOK

Blink

praktikum_gambar_1_sketch

sketch

Blink.ino

```
1 #include <DHT.h>
2
3 const byte dhtPin = 13;
4 #define DHTTYPE DHT11
5
6 DHT dht(dhtPin, DHTTYPE);
7
8 void setup() {
9   Serial.begin(115200);
10  dht.begin();
11  Serial.println("Memulai Sensor Lingkungan...");
12 }
13
14 void loop() {
15   delay(2500);
16
17   float temp = dht.readTemperature();
18   float hum = dht.readHumidity();
19
20   if (isnan(temp) || isnan(hum)) {
21     if (isnan(temp) || isnan(hum)) {
22       Serial.println("Gagal membaca data dari sensor DHT!");
23       return;
24     }
25     Serial.print("Suhu: ");
26     Serial.print(temp);
27     Serial.print(" Celcius | Kelembapan: ");
28     Serial.print(hum);
29     Serial.println(" %");
30 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM6')

New Line 115200 baud

Suhu: 31.40 C | LDR: 757
Kondisi Aman

Suhu: 31.40 C | LDR: 756
Kondisi Aman

NEW SKETCH

